UNIVERSITI TENAGA NASIONAL

The Energy University

CSEB5223 – SOFTWARE CONSTRUCTION & METHODS

FULL LAB CSEB5223

Student Learning Management System

GROUP KEJORA :

No.	NAME	STUDENT ID	CONTRIBUTION /100%
1	KHABILLAN A/L B. SIVASHANGKAR	SW01083321	100
2	WILFRED NIGEL A/L SHAWN TIMOTHY	SW01081451	100
3	MUHAMMAD ZULFADHLI BIN MUSTAPHA	SW01082103	100
4	NUR KHADIJAH AZIZ BINTI AZIZUL SHAHZALI	SW01083569	100

TABLE OF CONTENTS

1. Introduction

1.1 Project Overview

1.2 Objectives

2. Lab 1 & 2: Pre-set Software Construction

2.1 Use Case Diagram

2.2 Class Diagram

2.3 Documentation Formatting

2.4 Coding File Formatting

2.5 Naming Conventions (Dictionary)

2.6 Function and Methods Modularity

2.7 Continuous Integration Rules

3. Lab 3 & 4: Course Module Implementation

3.1 Course Input Implementation

3.2 Course Output Implementation

3.3 Analysis of Software Construction Best Practices

3.4 Challenges Faced During Development

4. Lab 5 & 6: Student Module Implementation

4.1 Student Input Implementation

4.2 Student Output Implementation

4.3 Analysis of Software Construction Best Practices

4.4 Challenges Faced During Development

5. Lab 7 & 8: Integrated System Implementation

5.1 Student Module (CRUD Operations)

5.2 Course Module (CRUD Operations)

5.3 Enrollment Module

5.4 API Integration (JSON Output)

5.5 Software Construction Best Practices

5.6 Challenges Faced During Development

6. Lab 9 & 10: Unit Testing for KSLMS

6.1 Test Case 1: Course Getter Method

6.2 Test Case 2: Student Getter Method

6.3 Test Case 3: Course Search Function

6.4 Test Case 4: Invalid Course Search

6.5 Test Case 5: Student Search Function

6.6 Test Case 6: Duplicate Enrollment Check

6.7 Software Construction Best Practices

6.8 Challenges Faced During Development

7. System Testing

7.1 System Test Case 1: Full System Flow

7.2 System Test Case 2: Multiple Enrollment

7.3 System Test Case 3: Invalid Enrollment Input

8. Conclusion

8.1 Summary

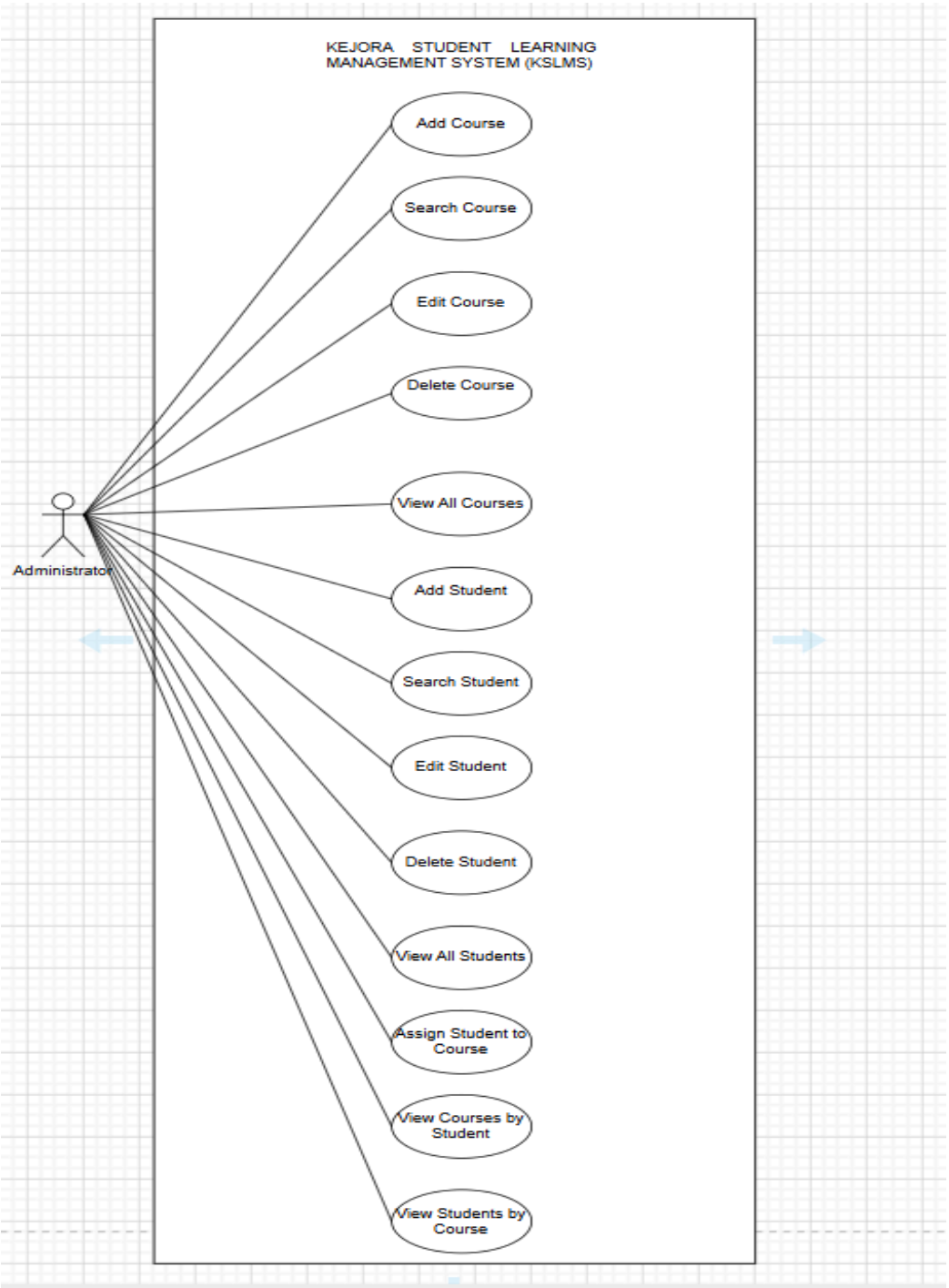
8.2 Key Learnings

8.3 Future Improvements

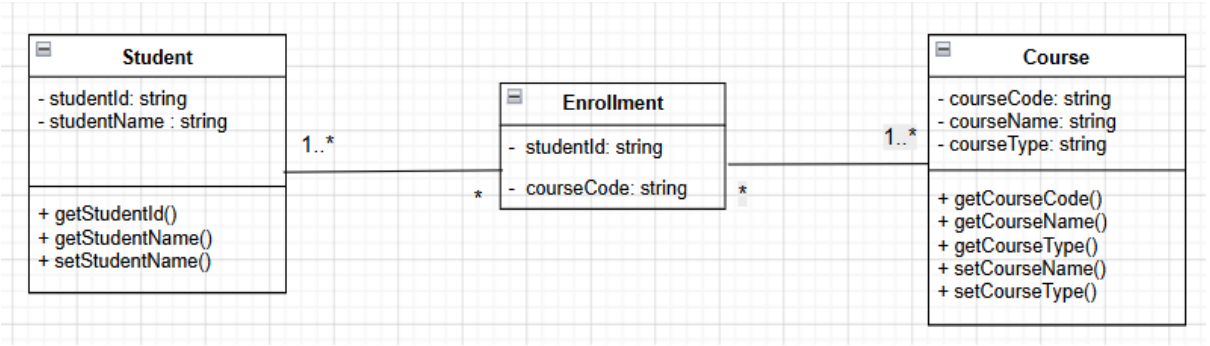
LAB 1&2 PRE-SET SOFTWARE CONSTRUCTION

1. Draft use case and class design of the overall SLMS

A. Use Case Diagram for Kejora Student Learning Management System



B. Class diagram for Kejora Student Learning Management System



2. Define documentation formatting

The documentation for KSLMS will follow a consistent academic report structure to ensure clarity and professionalism. The report will use:

- **Font: Times New Roman or Arial**
- **Font size: 13 and 14**
- **Line spacing: 1.5**
- **Text alignment: Justified**
- **Screenshots with labels and captions for explanation**

Each section of the documentation should clearly explain the design, implementation, testing, challenges, and software construction practices applied in the system.

3. Define coding file formatting

To maintain readability and consistency, the coding format for KSLMS will follow these rules:

- Indentation uses 4 spaces
- Opening braces are written on the same line for functions and classes
- Meaningful spacing is used between sections of code
- Comments are included where necessary to explain important logic
- Each class and controller should be stored in the correct Laravel directory
- Blade files should be separated according to module, such as **`courses`**, **`students`**, and **`enrollments.`**

This formatting standard helps keep the code clean, organized, and easier to debug and maintain.

4. Define naming conventions - dictionary

The naming convention in KSLMS is designed to be clear and self-explanatory. Classes use **PascalCase**, such as **Course**, **Student**, **CourseController**, and **EnrollmentController**. Variables and function names use **camelCase**, such as **studentId**, **courseCode**, **findStudentById()**, and **listCoursesByStudent()**. Blade files use

lowercase naming and descriptive names, such as create.blade.php, index.blade.php, and search_result.blade.php.

A simple naming dictionary for the system includes:

- **studentId** = unique identifier for a student
- **studentName** = full name of the student
- **courseCode** = unique identifier for a course
- **courseName** = full name of the course
- **courseType** = category of the course, such as core, elective, or university
- **enrollments** = array storing student-course relationships
- **getStudents()** = retrieve student list from session
- **getCourses()** = retrieve course list from session
- **saveEnrollments()** = save relationship data into the session

These conventions improve code readability and reduce confusion during development.

5. Define function and methods modularity

The KSLMS system is designed using a modular approach where each function and method is responsible for performing a single, well-defined task. This helps to improve code readability, maintainability, and reusability. In this system, separate controllers such as **CourseController**, **StudentController**, and **EnrollmentController** are used to handle different modules of the application. Each controller contains methods that

are logically grouped based on their functionality, such as creating, storing, searching, updating, and deleting data.

For example, in the Student module, methods like **getStudents()** and **saveStudents()** are used specifically for handling session-based data retrieval and storage. The method **findStudentIndexById()** is implemented to perform a linear search to locate a student based on their student ID. Similarly, in the Course module, functions such as **findCourseIndexByCode()** are used to locate courses efficiently. In the Enrollment module, methods such as **enrollmentExists()** are used to check for duplicate relationships, while **listCoursesByStudent()** and **listStudentsByCourse()** handle the retrieval of relationships between students and courses.

This modular design ensures that each function has a clear responsibility and avoids mixing multiple logics in a single method. It also makes debugging easier because errors can be traced to specific functions. Furthermore, it improves scalability, as new features can be added without affecting existing modules. Overall, applying modularity in KSLMS ensures a clean structure, reduces code duplication, and enhances system reliability.

6. Define continuous integration rules

To ensure consistent development and reduce errors during the construction of KSLMS, a set of continuous integration rules is defined. Firstly, developers should commit their code regularly to the GitHub

repository after completing small and meaningful features, such as implementing course creation, student management, or enrollment functionality. Each commit should include a clear and descriptive message to indicate the changes made, which helps in tracking the development progress.

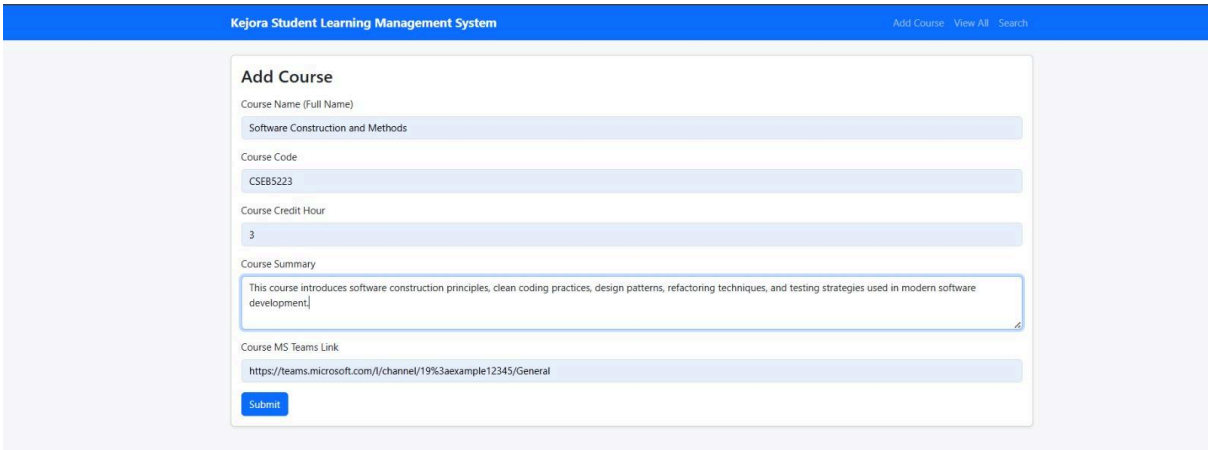
Secondly, testing should be performed before each commit to ensure that newly added features do not break existing functionalities. For example, after implementing the enrollment module, the developer should verify that the course and student modules still function correctly. This helps maintain system stability throughout the development process. In addition, developers should follow the predefined coding standards, naming conventions, and formatting rules consistently to ensure uniformity across all files.

Furthermore, version control practices such as maintaining a clean project structure and avoiding unnecessary files in the repository should be followed. Any updates or modifications should not affect the integration between modules, especially the relationship between courses, students, and enrollments. Screenshots and documentation should also be updated alongside code changes to reflect the latest system implementation.

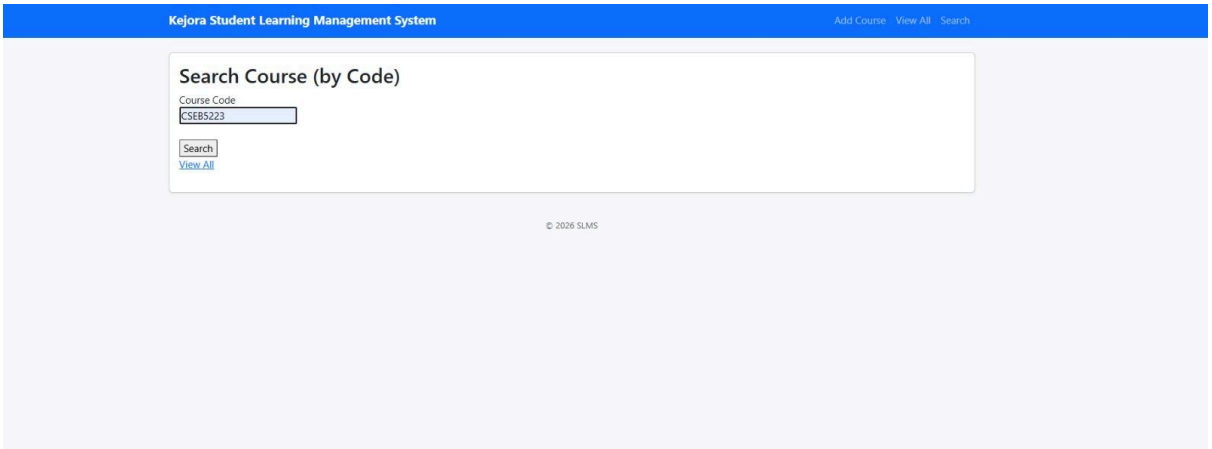
Finally, the system should be tested as a whole after integrating all modules to ensure that all components work together seamlessly. By following these continuous integration rules, the development process becomes more organized, collaborative, and less prone to errors, ensuring a higher quality final system.

LAB 3&4 Screenshots of Inputs and Outputs

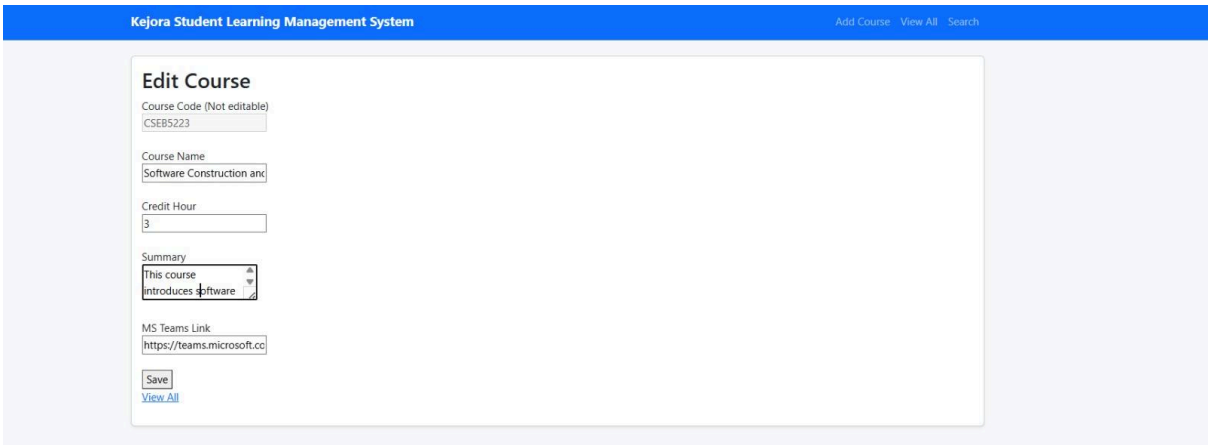
Course Input Implementation



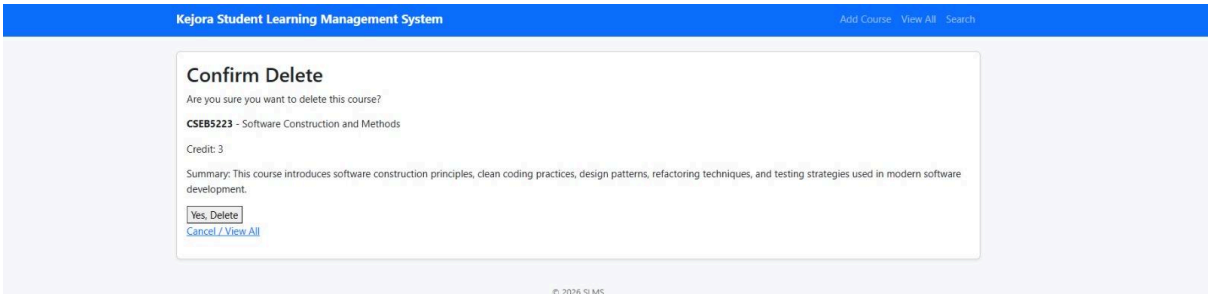
Add course (adding description)



Search course

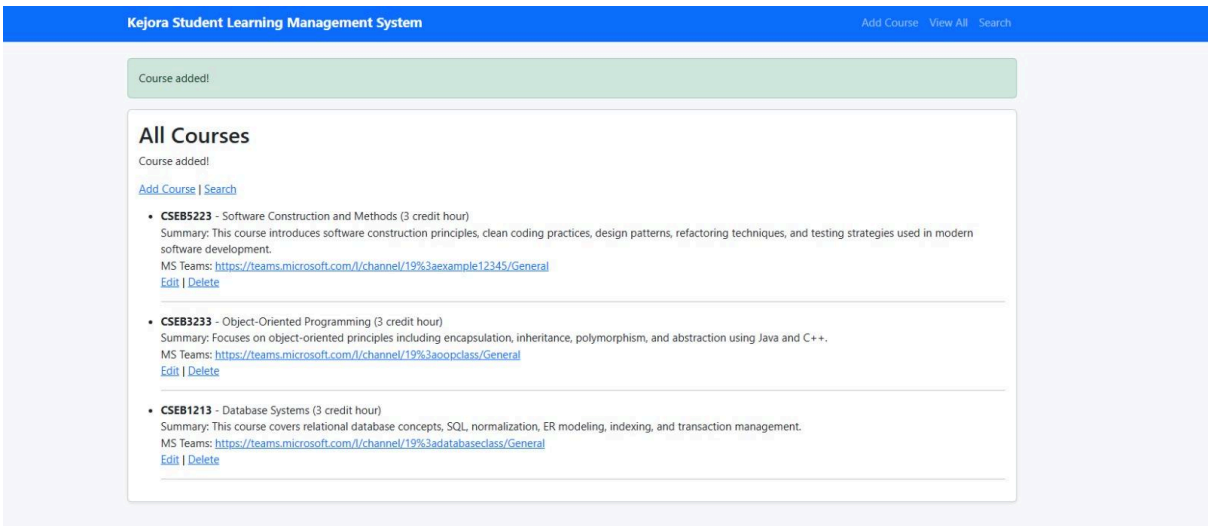


Edit course

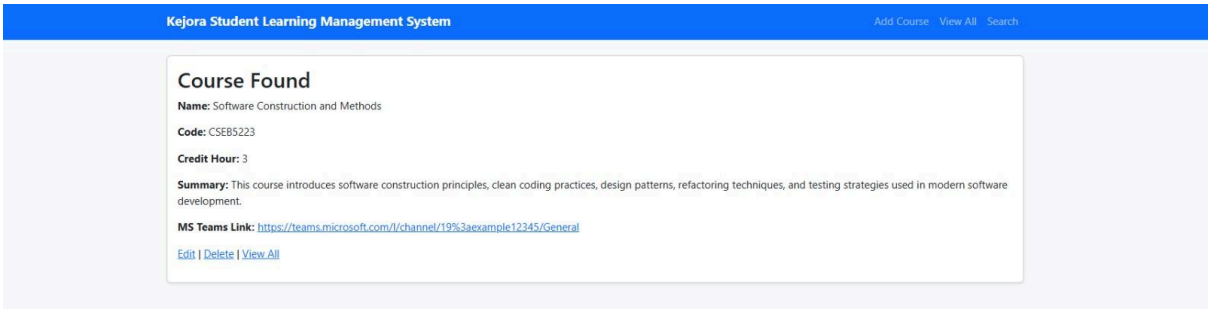


Delete course

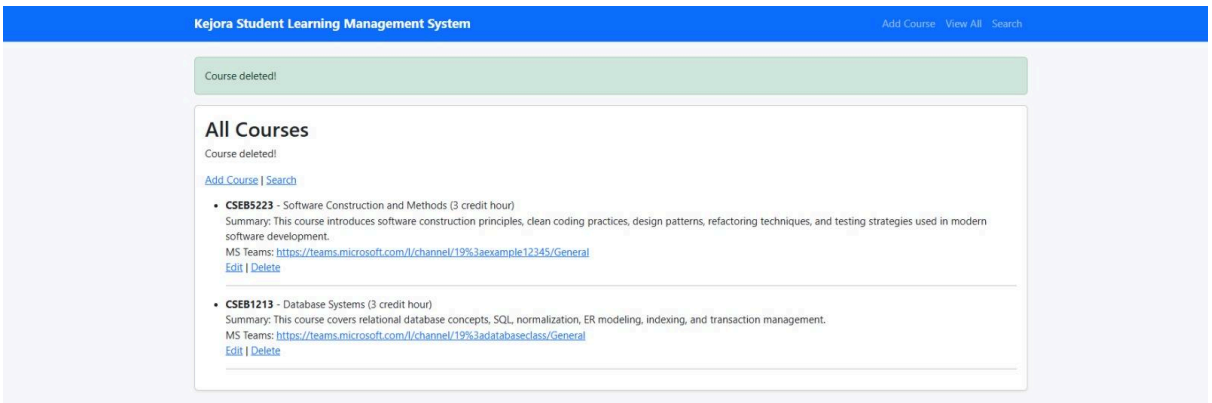
2. Course Output Implementation



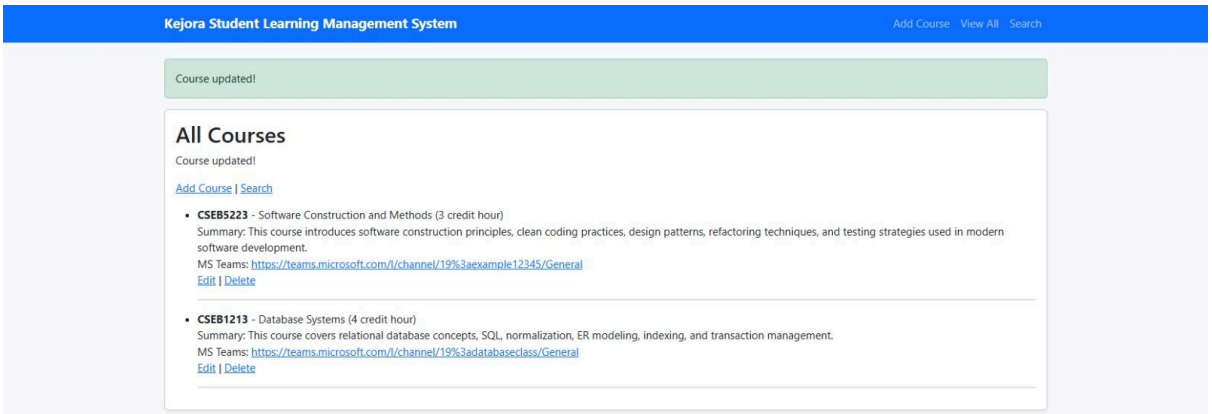
View course (add description, Navigation)



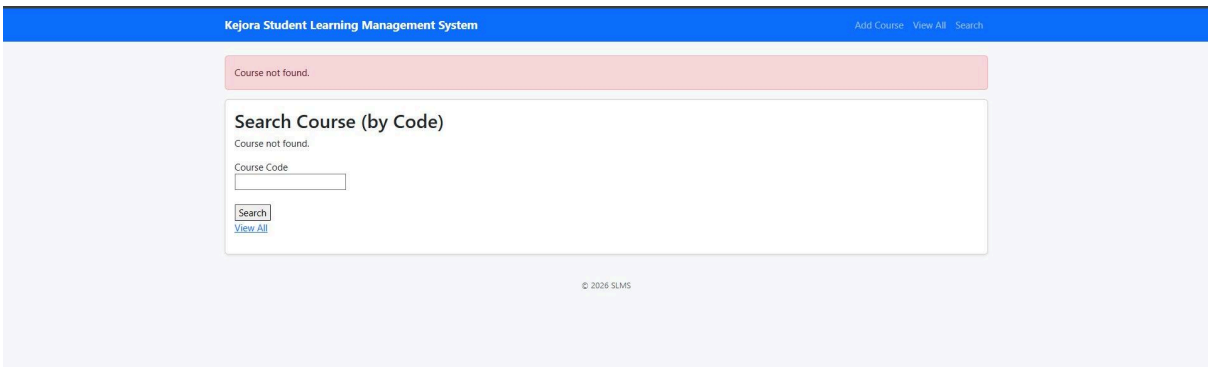
View the specific Course after the search



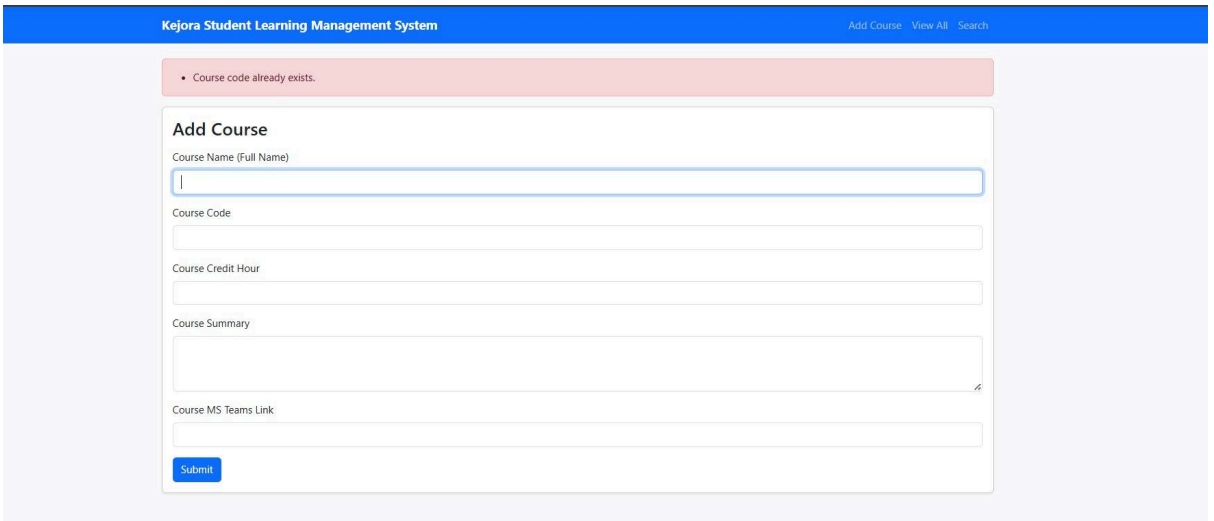
View the course after deleting a course



View Course after edit (Edit Database course Credit hour from 3 to 4)



Course not invalid



Course Duplication is not allowed

Analysis of Software Construction Best Practices

Several best practices for software construction were used in the development of the Kejora Student Learning Management System (KSLMS). The Laravel framework's Model-View-Controller (MVC) design, which helps divide application logic, user interface, and data processing, is

followed by the system. This enhances the maintainability and organisation of the code.

The Course class was created to apply object-oriented programming (OOP) principles. Private variables are used to contain characteristics such as the course name, course code, credit hours, summary, and MS Teams link. Better data integrity was ensured by controlling access and alteration of certain characteristics through the use of getter and setter methods.

Furthermore, a linear search technique was used to find courses using the course code. To avoid problems like duplicate course entries, proper validation and error handling and array out-of-bound errors were implemented.

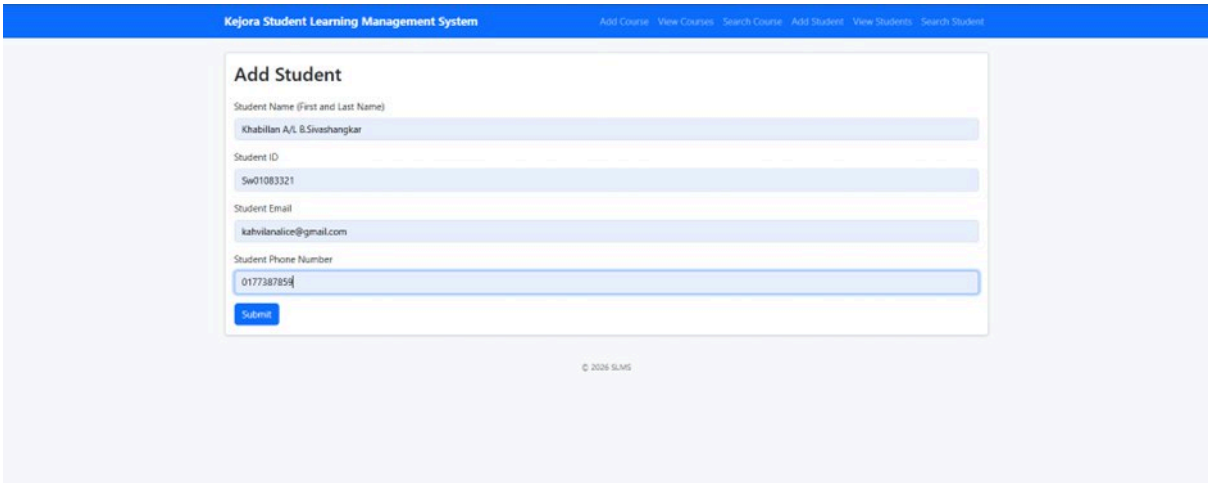
Challenges Faced During Development

Several difficulties occurred throughout the KSLMS's development. Implementing object storage without a database was one of the challenges. Rather, an array of course objects was stored using session-based storage. Making sure the linear search method could securely find courses without resulting in array index errors presented another difficulty.

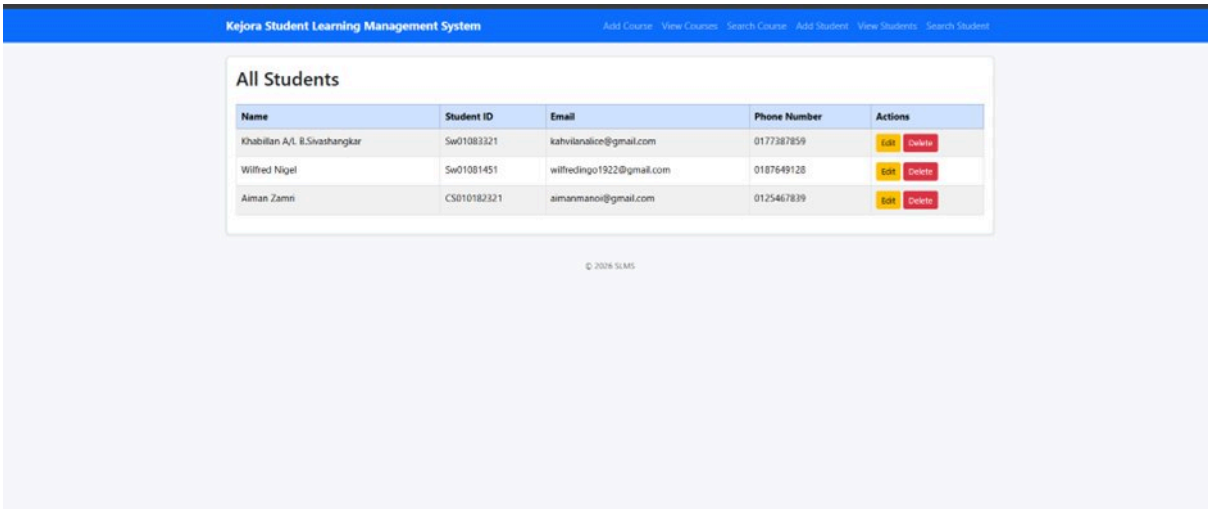
Additionally, knowing Laravel's template inheritance mechanism (@extends and @yield) was necessary for creating a reusable layout utilising Blade templates. To make sure that system features like search, edit, and delete operated properly, proper testing and debugging were required.

LAB 5&6 Screenshots of Inputs and Outputs

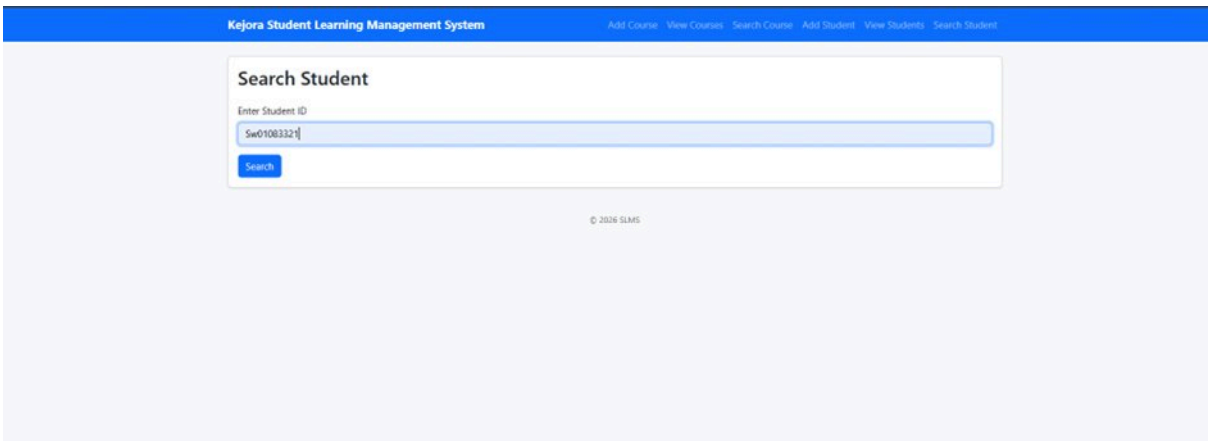
A. Input



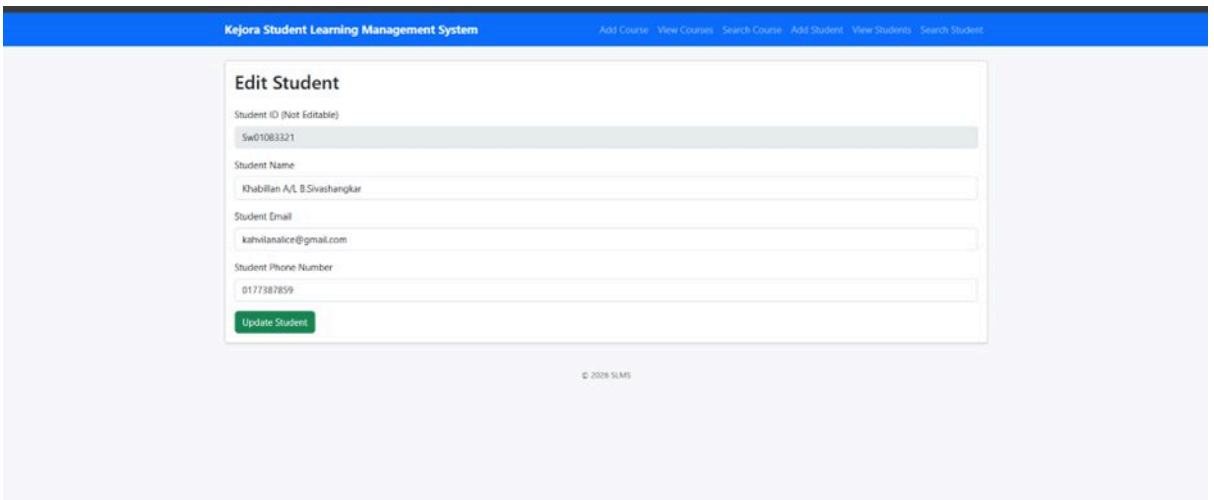
Add Student



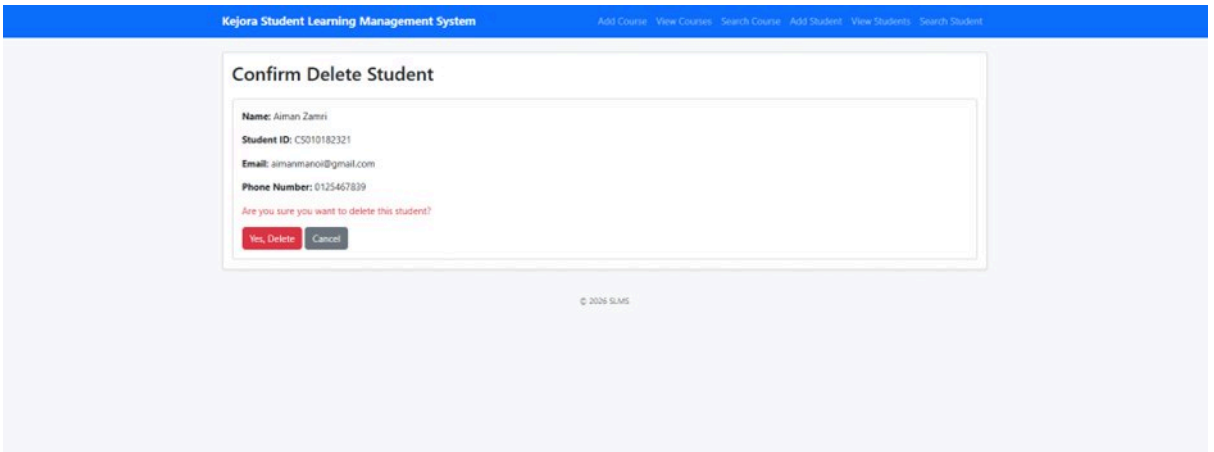
View Students



Search Student

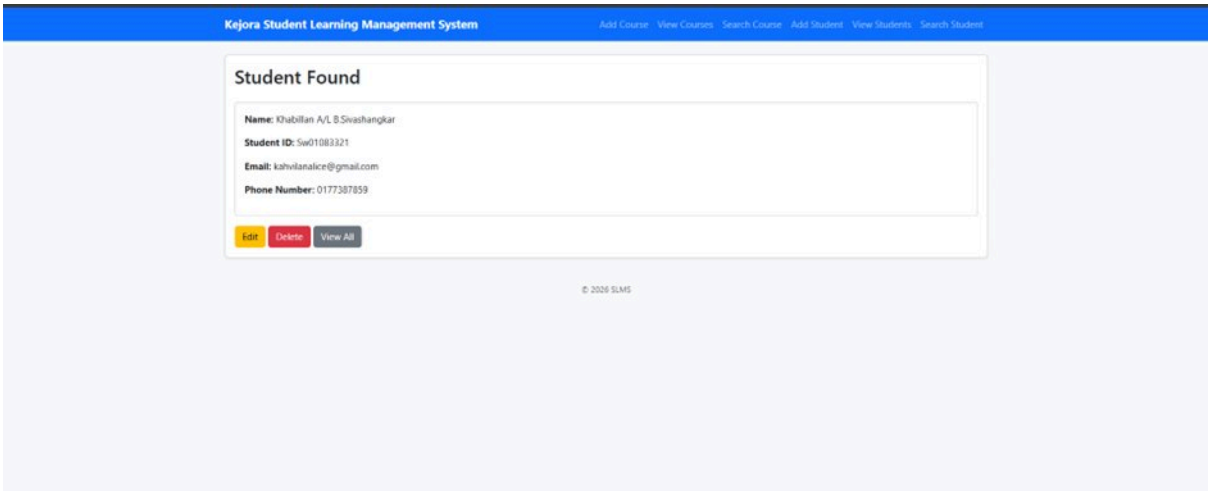


Edit Student

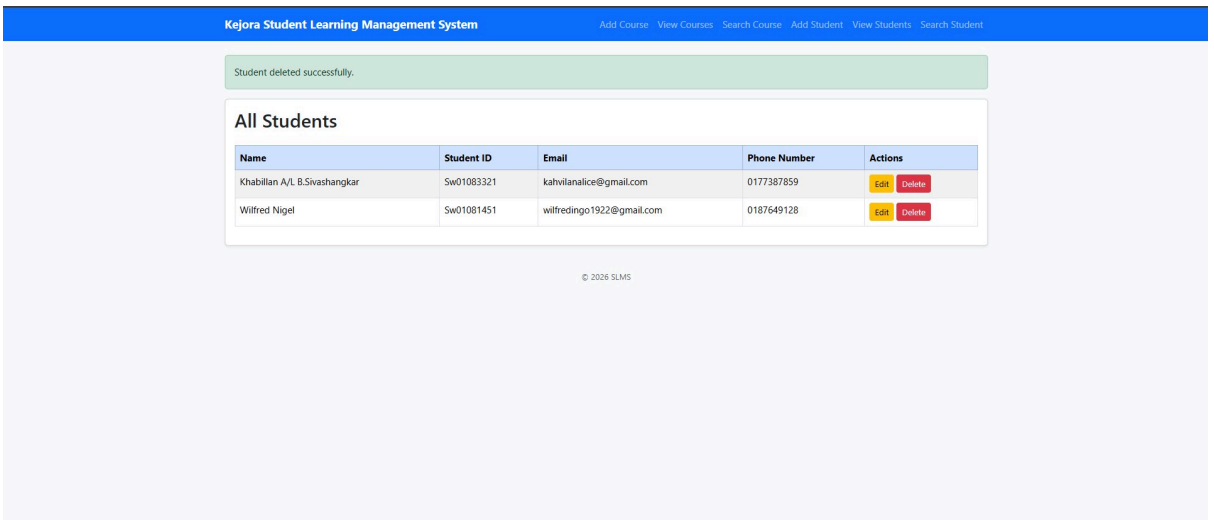


Delete Student

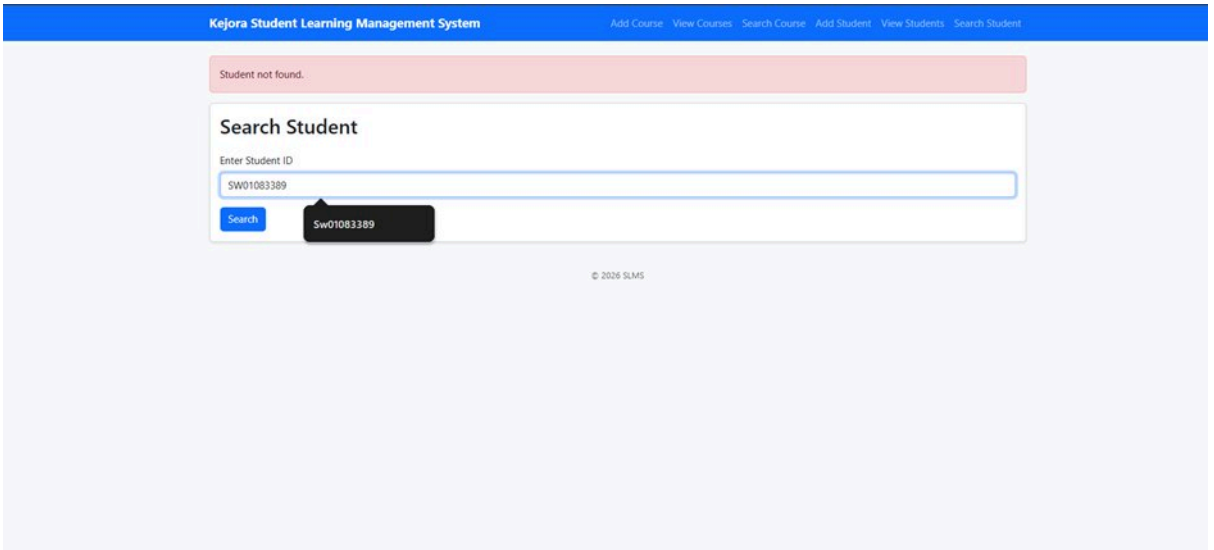
B. Output



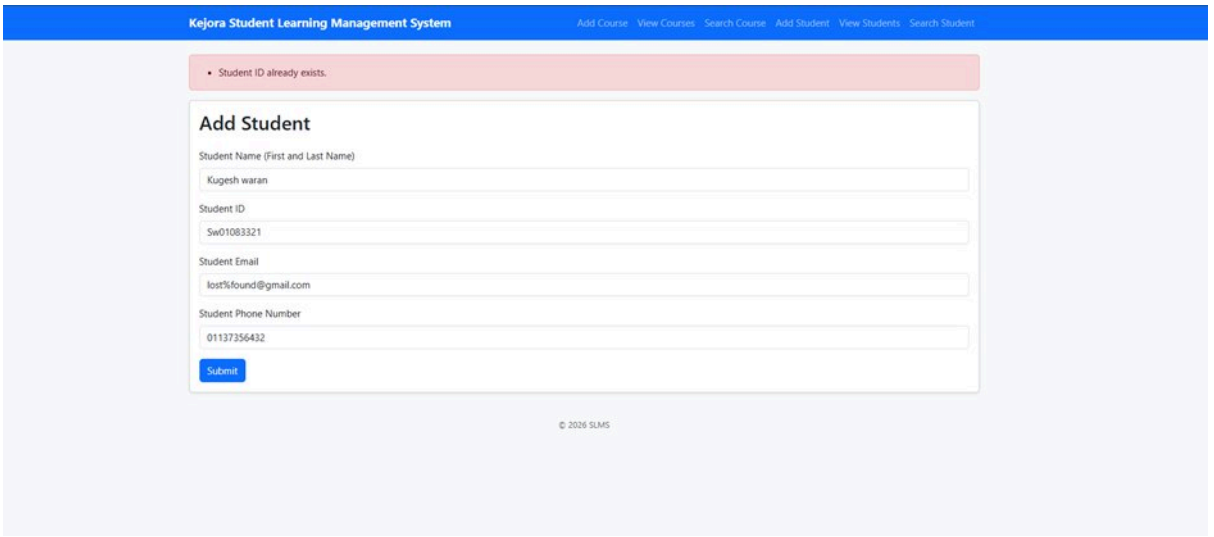
View Student after searching for the student



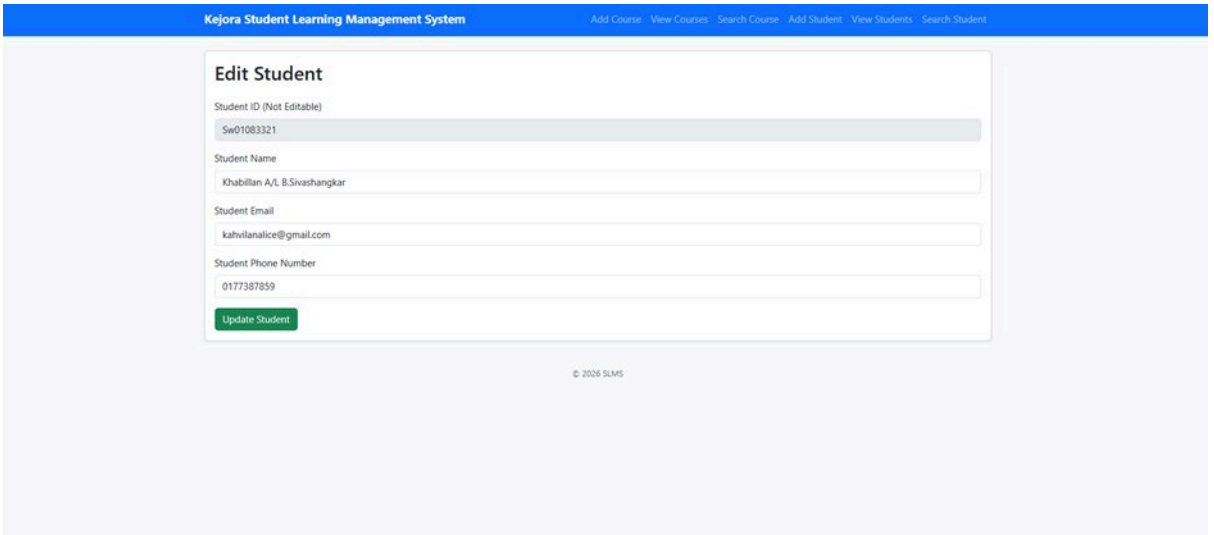
View all students after a student has been deleted



Invalid Student ID



Student cannot have same student id with registered student when registering (avoid duplication)



Student id cannot be changed in order to prevent id duplication or student using the same id

Analysis of Software Construction Best Practices

In developing the Kejora Student Learning Management System (KSLMS), several software construction best practices were applied to ensure that the system is well-structured, maintainable, and reliable. Firstly, the system follows the Model-View-Controller (MVC) architecture provided by the Laravel framework, which separates the application into Models, Views, and Controllers. This improves code organization and makes the system easier to maintain and extend. In addition, Object-Oriented Programming (OOP) principles were implemented through the creation of the Course and Student classes. Each class contains private attributes, and access to these attributes is controlled through getter and setter methods. This demonstrates encapsulation, which helps protect data integrity and prevents unauthorized modification. For example, the Course Code and Student ID are designed to be immutable to ensure consistency within the system.

Furthermore, code reusability and modularity were achieved through the use of Blade template inheritance. A common layout file was created and reused across multiple pages using Laravel's @extends and @yield directives, reducing code duplication and ensuring a consistent user interface. Input validation and error handling were also implemented using Laravel's built-in validation features to ensure that only valid data is accepted. The system provides user-friendly error messages such as "Course not found" and "Student not found," which enhances the overall user experience and system reliability.

Another important practice applied is safe array handling. Since the system uses session-based storage instead of a database, checks such as isset() are performed before accessing array elements to prevent array

out-of-bound errors. Additionally, a linear search algorithm was implemented to locate courses and students based on their unique identifiers. The algorithm safely iterates through the array and returns appropriate results when data is found or not found, ensuring stable and reliable system performance.

Challenges Faced During Development

During the development of KSLMS, several challenges were encountered. One of the main challenges was implementing data storage without using a database. Instead of using MySQL, session-based storage was used to store arrays of Course and Student objects. Managing and updating these objects required careful handling to ensure data consistency across different operations.

Another challenge was implementing the linear search function while ensuring that no array index errors occur. Proper checks, such as verifying whether the returned index exists using isset(), were necessary before accessing the array.

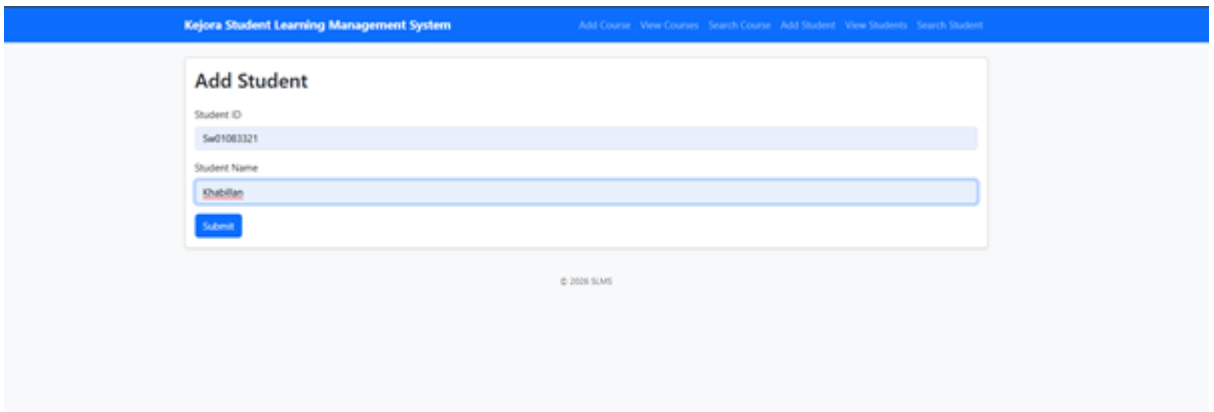
Additionally, designing the system using Blade template inheritance required a clear understanding of how Laravel handles layouts and content sections. Mistakes in placing directives such as @extends, @section, and @yield initially caused layout issues, which needed to be debugged and corrected.

Handling form validation and user input errors was also challenging. The system needed to ensure that all required fields were filled correctly while providing clear and meaningful feedback to users.

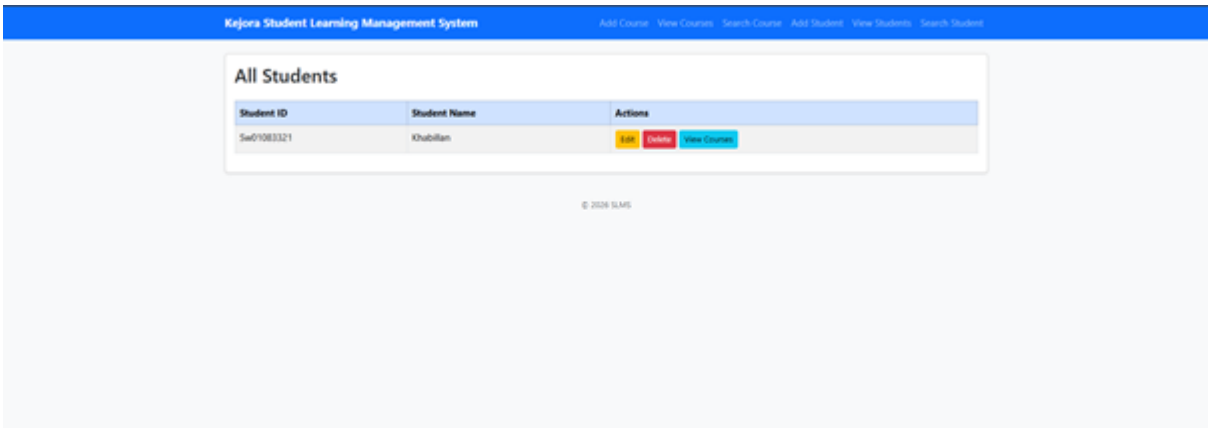
Finally, integrating both Course and Student modules into a single system required consistent routing, navigation design, and reuse of layout components to ensure a smooth and seamless user experience across different modules.

LAB 7&8 Screenshots of Inputs and Outputs

Input Students



Adding Student



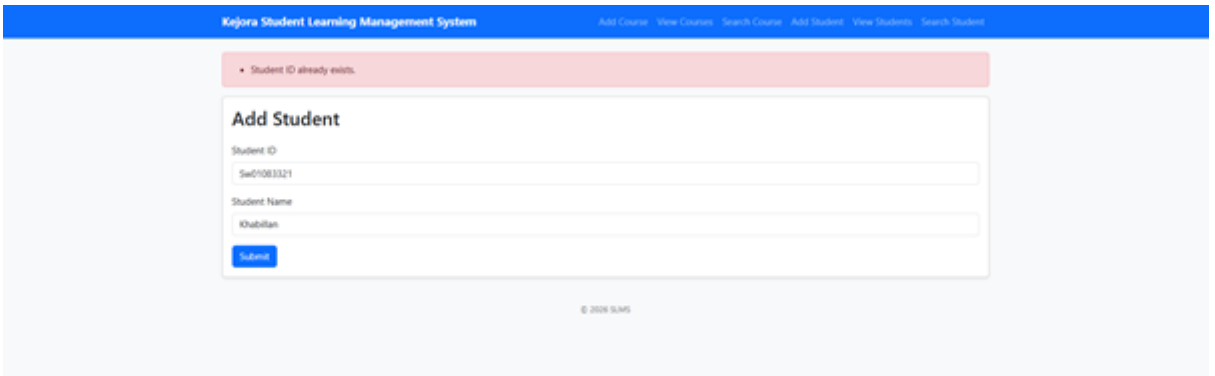
View Student

Search Student

Delete student

Edit Student

Output Students

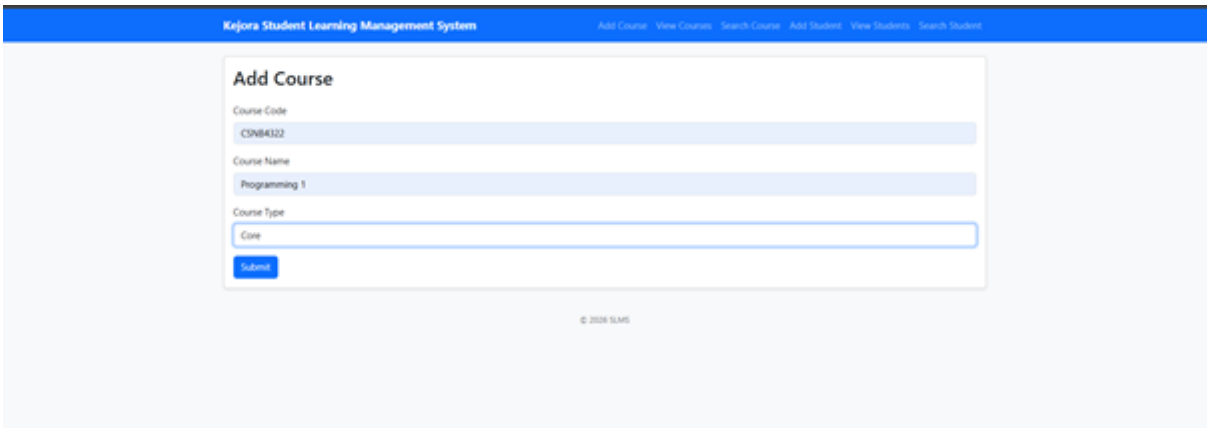


Student duplication is not allowed (Fixed Student ID)

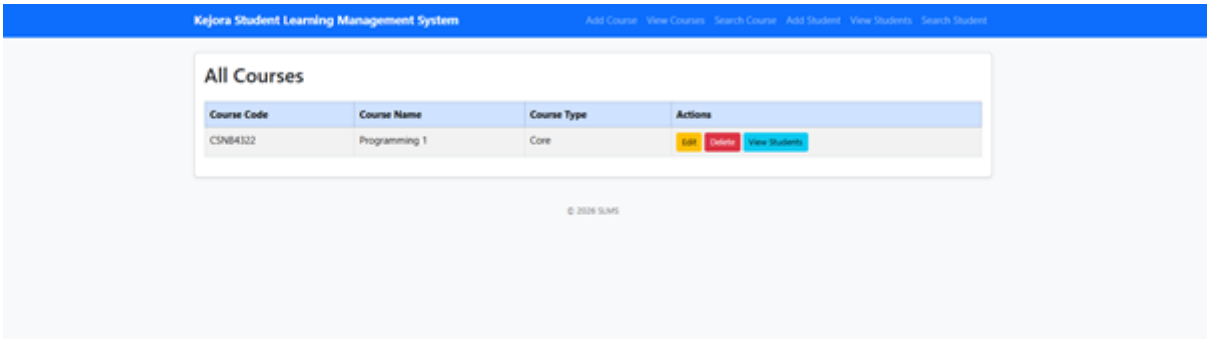
Student that is not registered will not be able to be searched since system doesn't recognize the ID

When editing, the student ID column can't be edited, so that the system will block any duplication id that are used when adding new students in the system

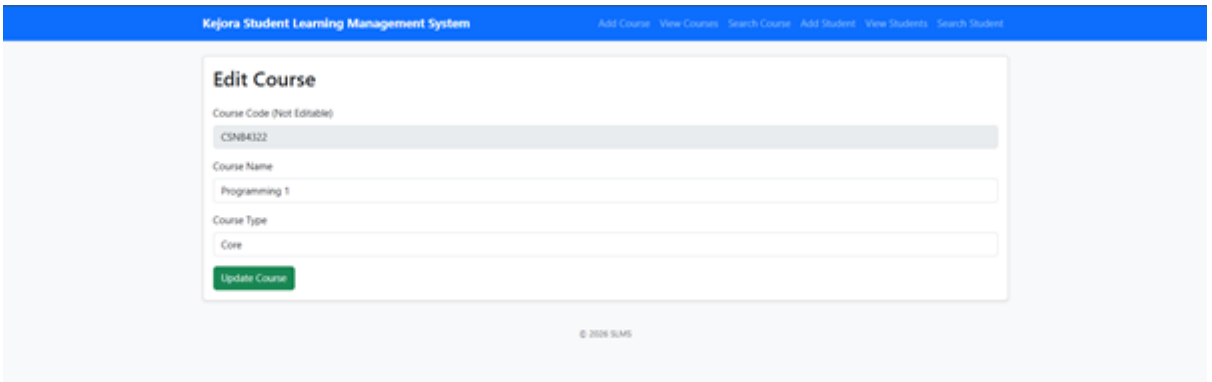
Input Courses



Adding a course



View Course

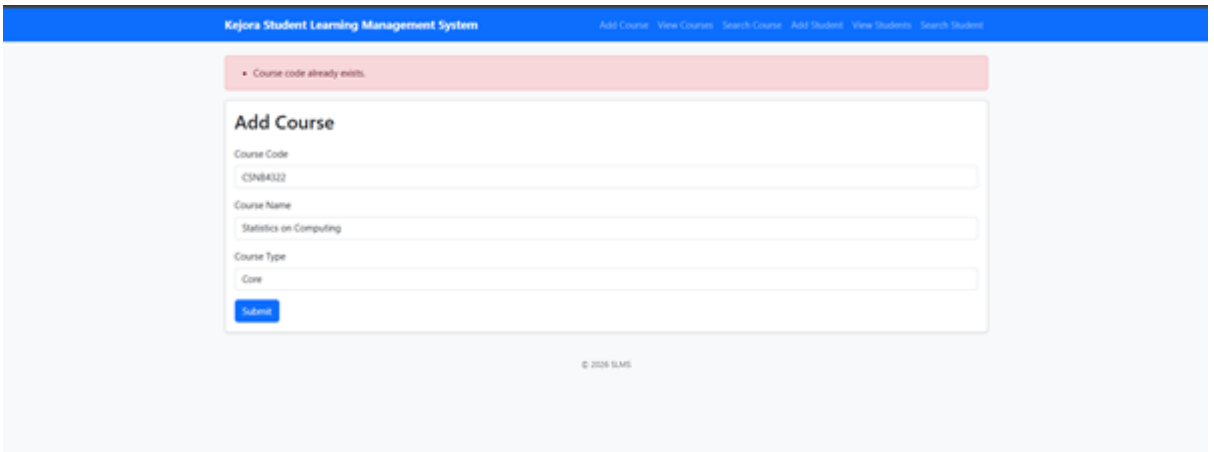


Edit course

Search Course

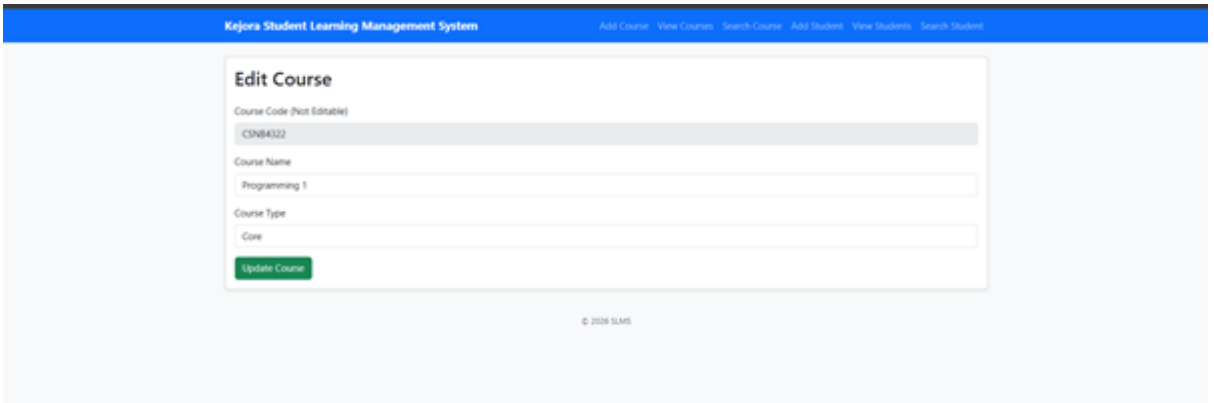
Delete Course

<u>Output Courses</u>



System doesn’t allow course code duplication

Course code that is not added can’t be searched by the system since the code is non-existence



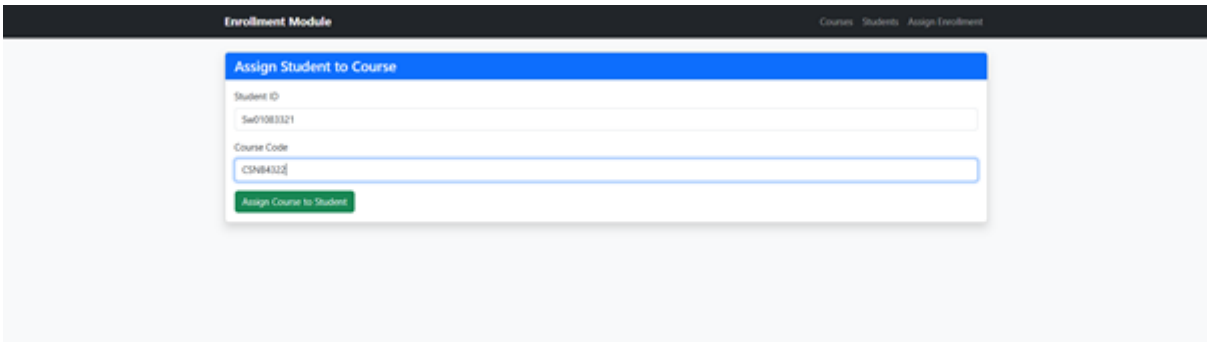
Course code is not editable when it's added in the system to avoid duplication when adding a course

API (Output)

Json key for the course that is registered in the system

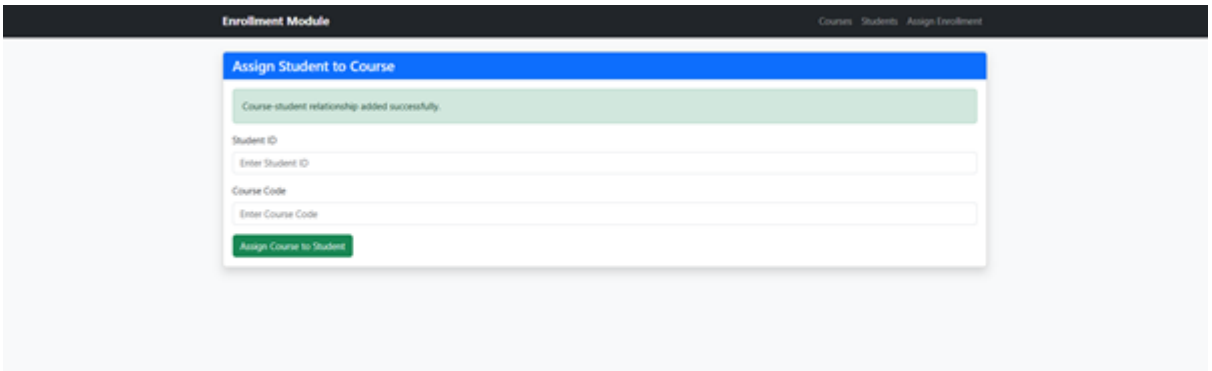
Json key for the course that is registered in the system

Input Enrollment

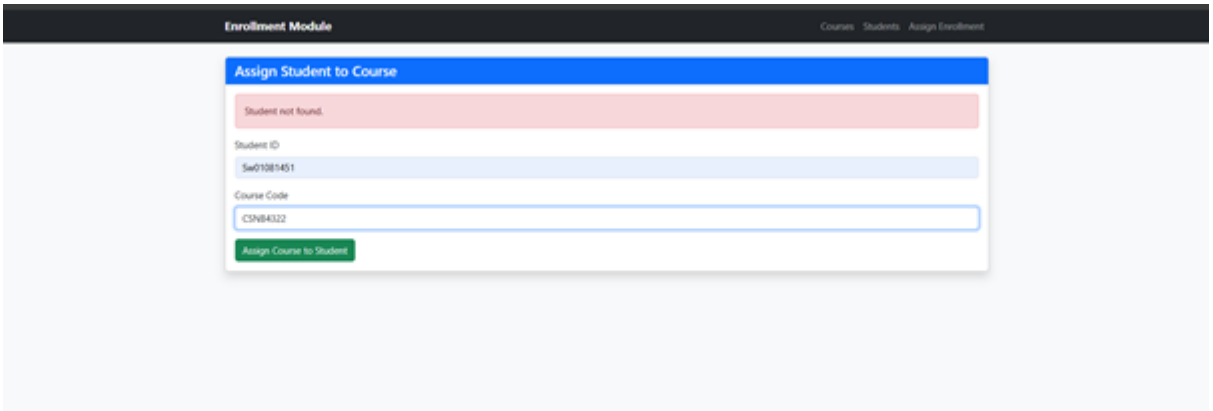


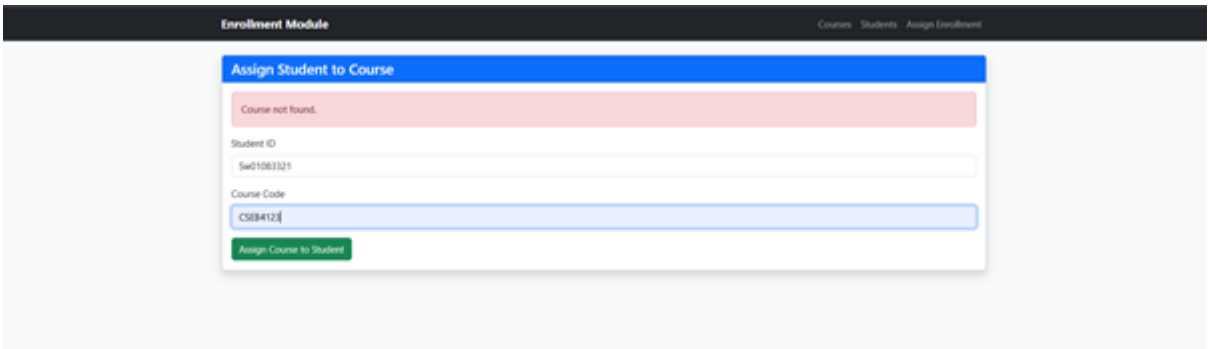
-

Output Enrollment



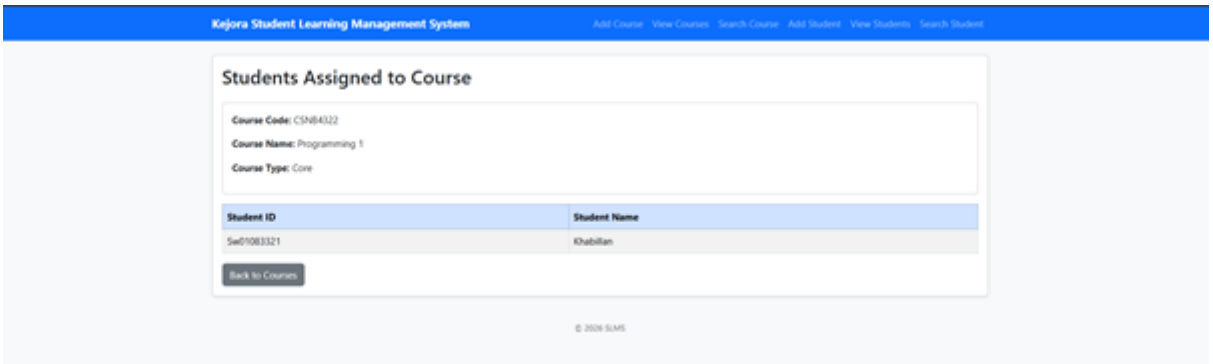
Successful when both code and student id is registered, API able to fetch the id and code and send it to the enrollment section.





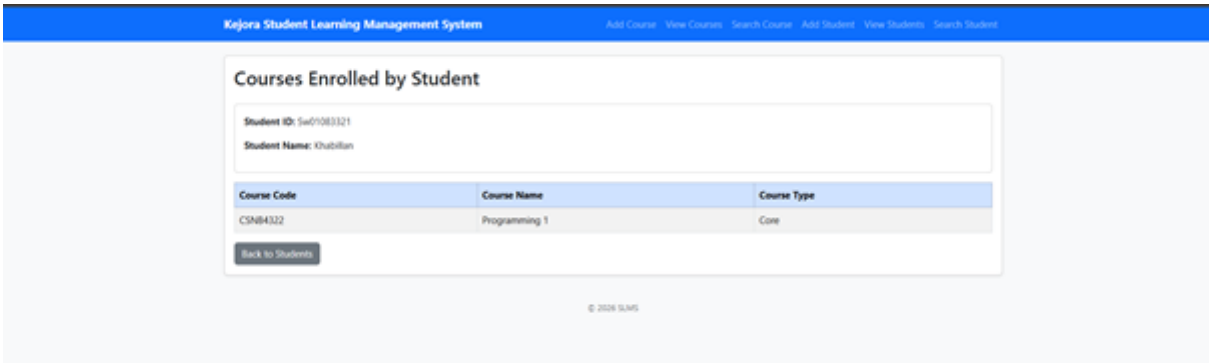
Unsuccessful if the API didn't receive one of the student IDs or course codes because it's not registered, and the API cannot fetch the data and give it to the enrollment section.

<u>Student and course enrolled (output)</u>



-

Successfully enrolled_(From Course view)



Successfully enrolled (From Student View)

SOFTWARE CONSTRUCTION BEST PRACTICES

In developing the Kejora Student Learning Management System (KSLMS), several software construction best practices were applied to ensure the system is well-structured, maintainable, and reliable. The system adopts a modular design approach, dividing functionalities into separate components such as the Student Module, Course Module, and Enrollment Module, allowing each module to operate independently and making the system easier to maintain, debug, and extend.

In addition, Object-Oriented Programming (OOP) principles were implemented through classes like Student and Course, where private attributes such as Student ID and Course Code are protected using getter and setter methods, ensuring encapsulation and safeguarding data integrity. Proper data validation and error handling mechanisms were also incorporated to ensure that only valid data is processed, including enforcing unique Student IDs and Course Codes, handling invalid inputs, and providing clear and user-friendly error messages to improve user experience. To maintain data consistency and integrity, critical attributes are designed to be non-editable after creation, preventing duplication and ensuring that each record remains unique throughout the system lifecycle.

Furthermore, API integration is utilised to enable communication between modules, particularly in the enrollment process, where data is exchanged in JSON format to ensure efficient and accurate data transfer. The system also implements a linear search algorithm to locate students and courses based on their identifiers, which is suitable for small-scale systems and ensures stable performance. Overall, maintainability and reusability are achieved through clean coding practices and well-structured functions, allowing developers to easily update or enhance the system without affecting existing functionalities.

CHALLENGES FACED DURING DEVELOPMENT

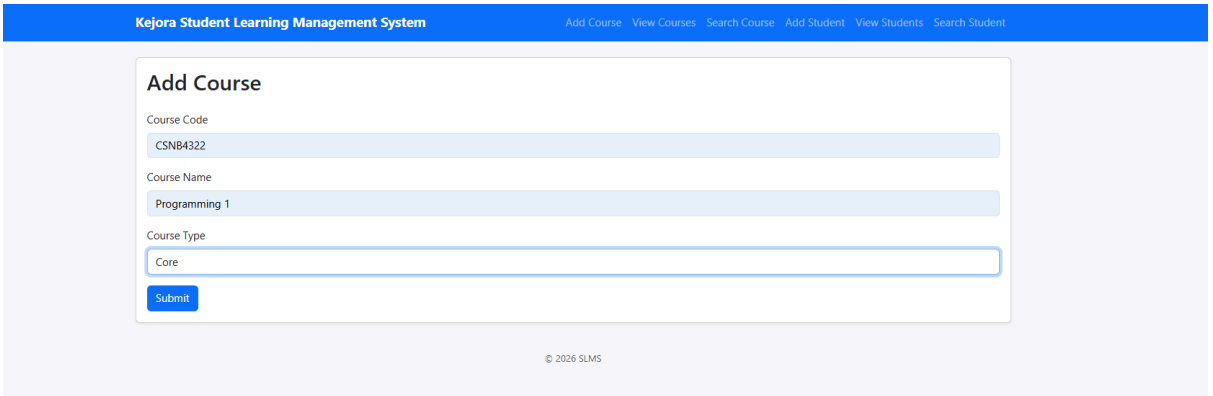
During the development of the Kejora Student Learning Management System (KSLMS), several challenges were encountered that required careful problem-solving and debugging.

One of the primary challenges was preventing data duplication, particularly ensuring that Student IDs and Course Codes remain unique, which required strict validation rules and the decision to make these fields non-editable after creation. Another challenge involved handling invalid data, where the system needed to verify the existence of students and courses before performing operations such as search, edit, or enrollment, while also ensuring that errors do not cause system crashes. The enrollment process introduced additional complexity, as it depends on both valid Student IDs and Course Codes, requiring proper validation and accurate data retrieval through API integration before allowing enrollment to proceed. Ensuring smooth data communication between modules was also challenging, especially since the system relies on API and JSON-based data exchange, which must be accurately transferred and interpreted across different components. In addition, managing CRUD (Create, Read, Update, Delete) operations without affecting system stability required careful handling to maintain data integrity during updates or deletions.

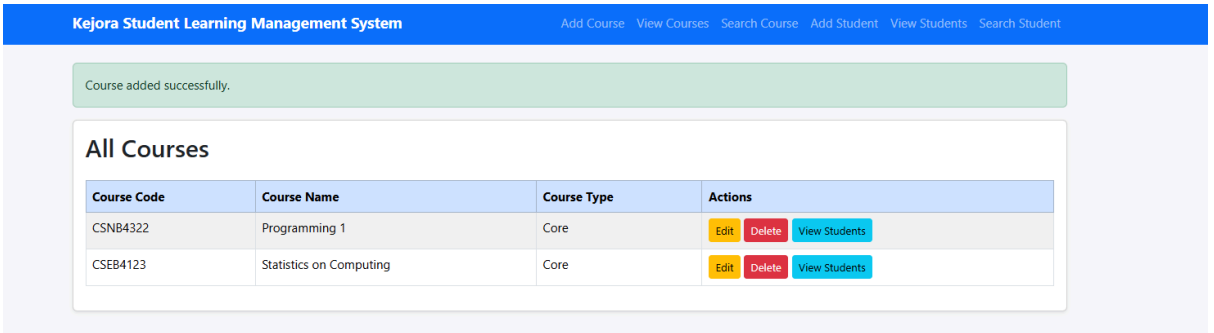
Balancing strong validation with a user-friendly interface was another concern, as the system needed to prevent errors while still providing a smooth and intuitive user experience with clear feedback. Finally, maintaining overall system stability while integrating multiple modules and features requires continuous testing and debugging to ensure that all functionalities work seamlessly together.

LAB 9&10 UNIT TESTING FOR KSLMS

UNIT Test Case 1 : Course Getter Method

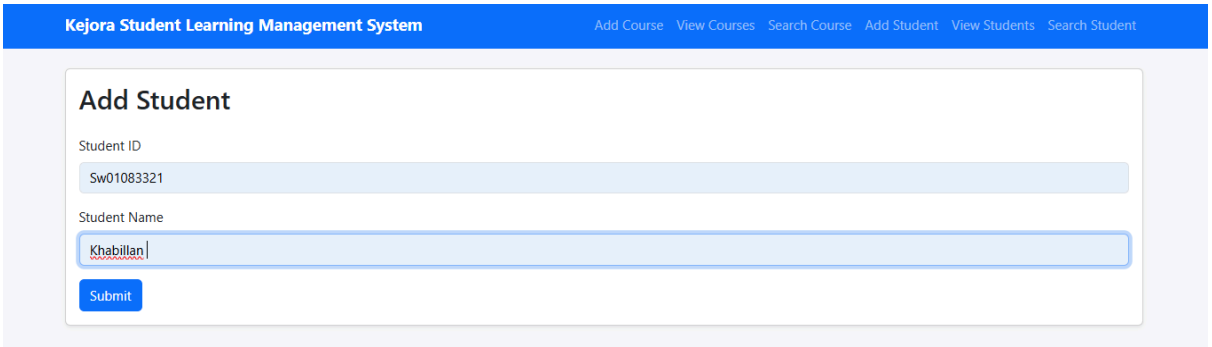


Add course

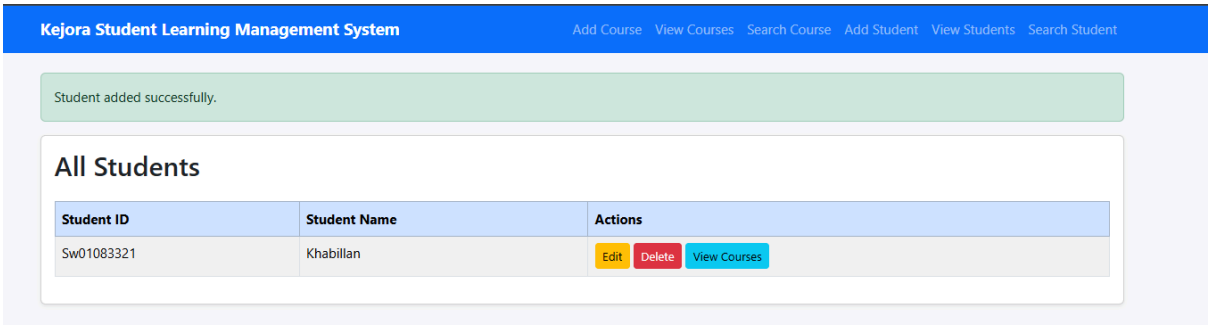


Add Course Page will be navigated to list of courses page after successfully adding course

UNIT Test Case 2: Student Getter Method

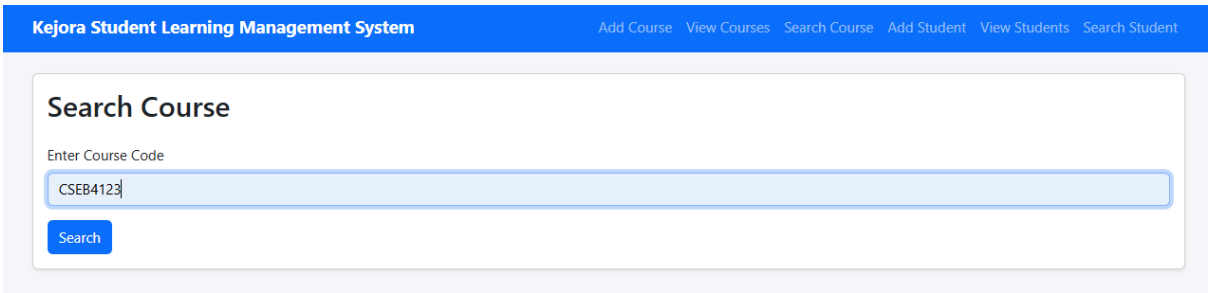


Add Student

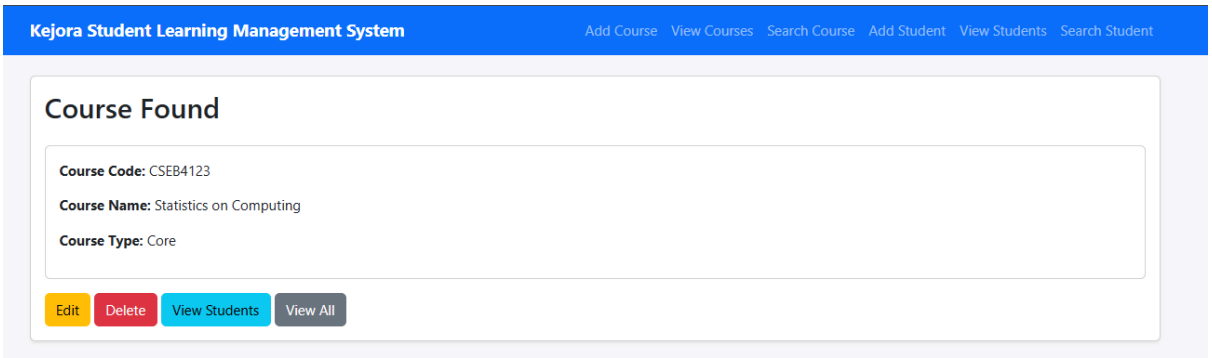


Add Student Page will be navigated to list of students page after successfully adding course

UNIT Test Case 3: Course Search Function

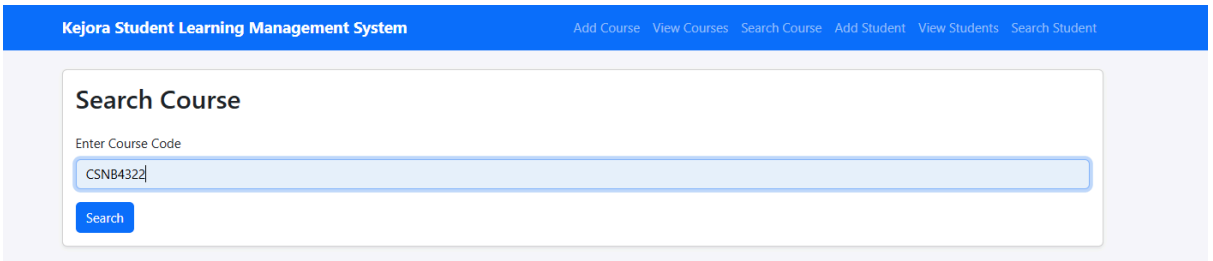


Search Course

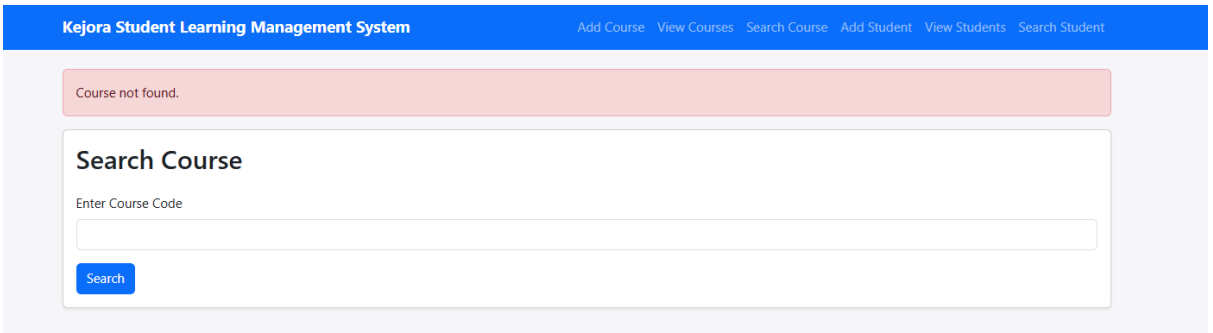


Course is successfully found because its registered in the system

UNIT Test Case 4: Invalid course search

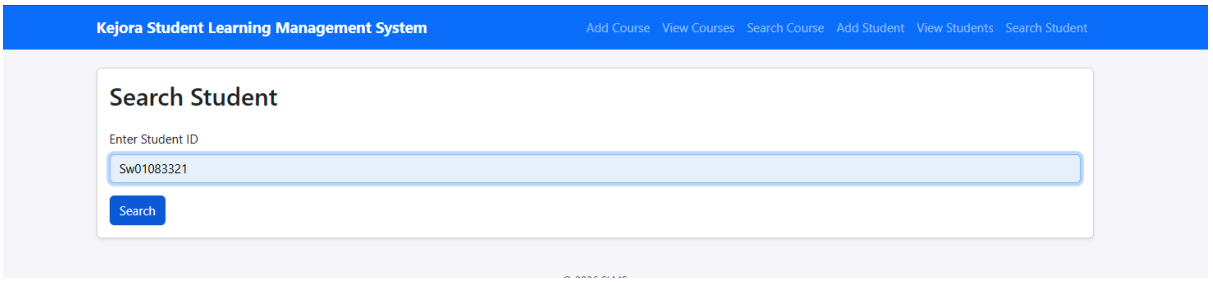


Search Course

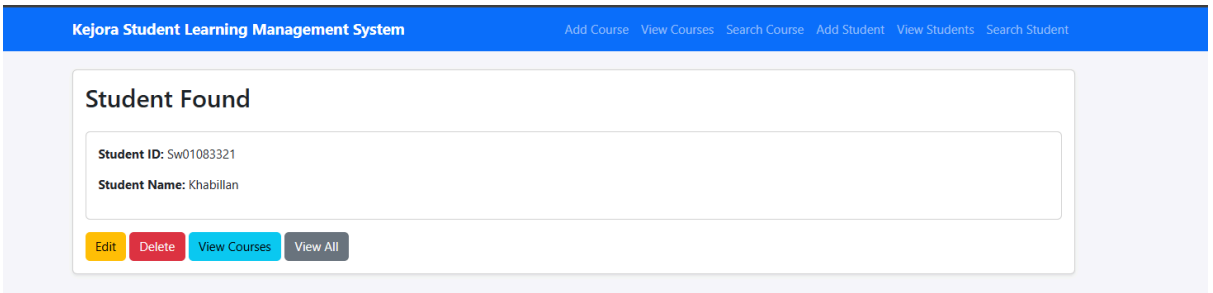


Course not found because its not registered in the system

UNIT Test Case 5: Student Search Function



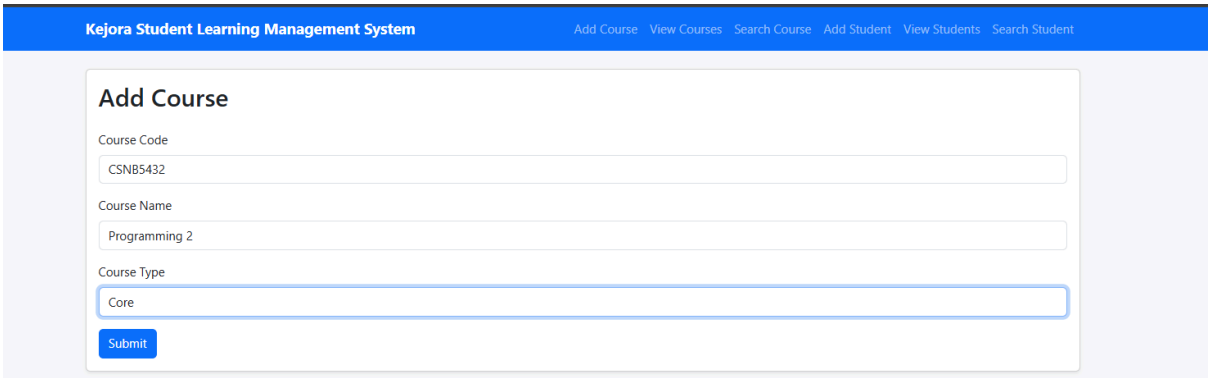
Search Student



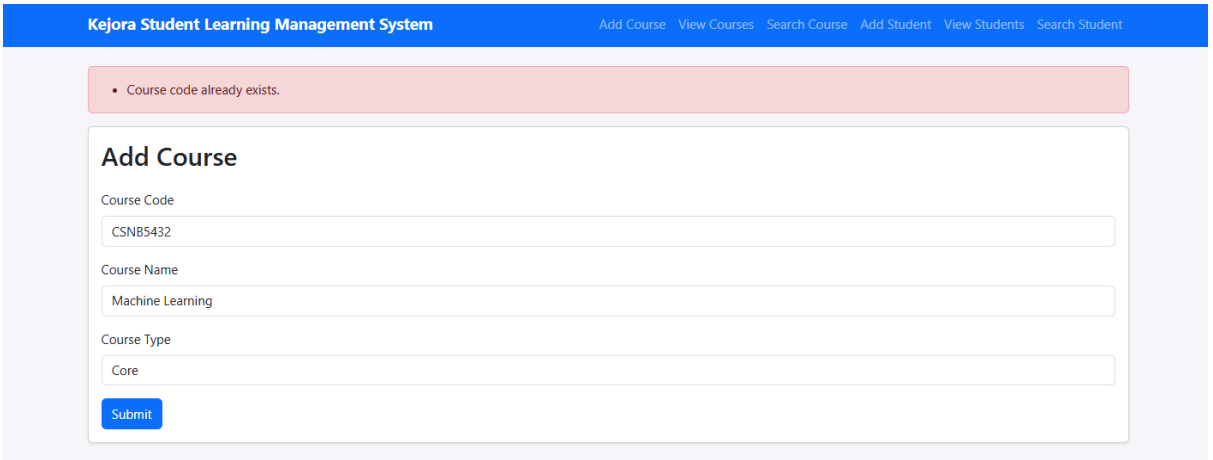
Student is Found because the student is registered to the system

UNIT Test Case 6: Duplicate Enrollment Check Both Student and Course

Course

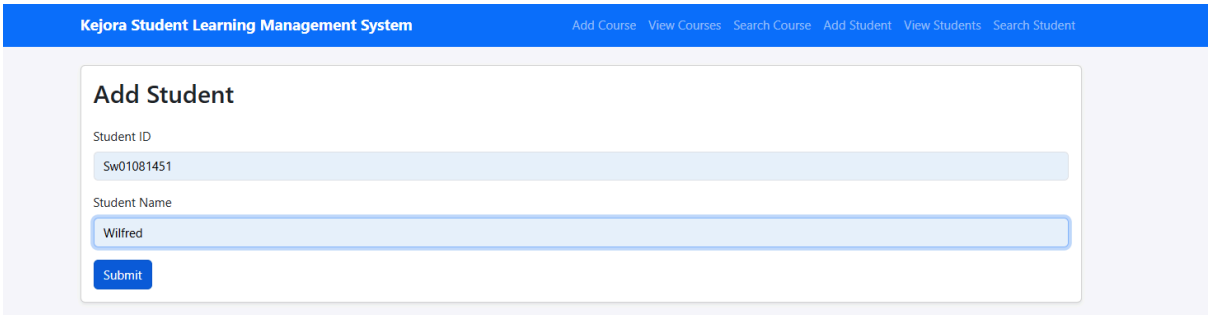


Adding a Course

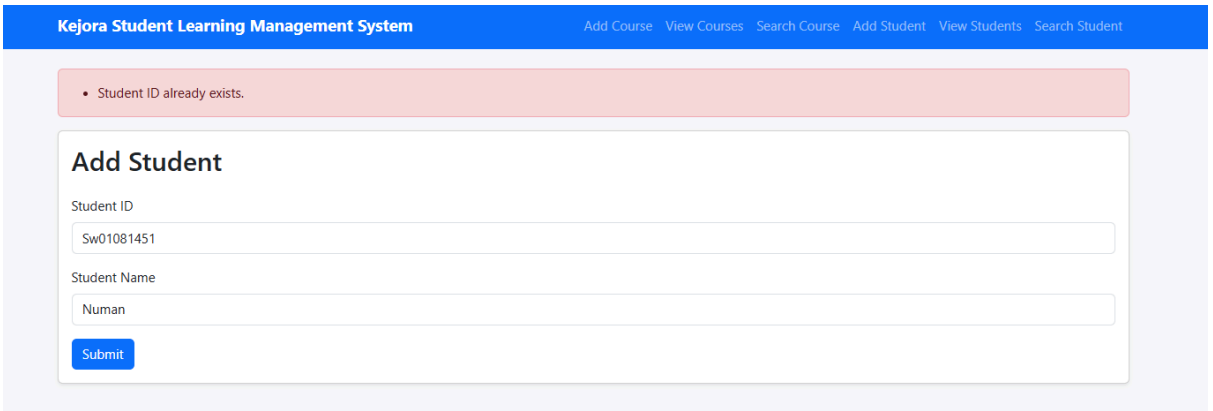


Adding a new course using the same course code that is used at another course will be denied by the system because of duplication

Student



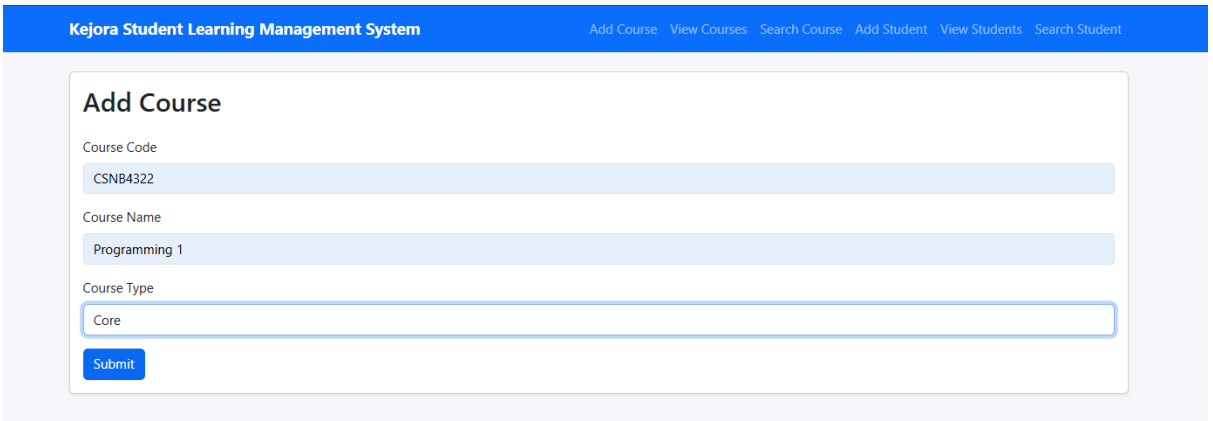
Adding a Student



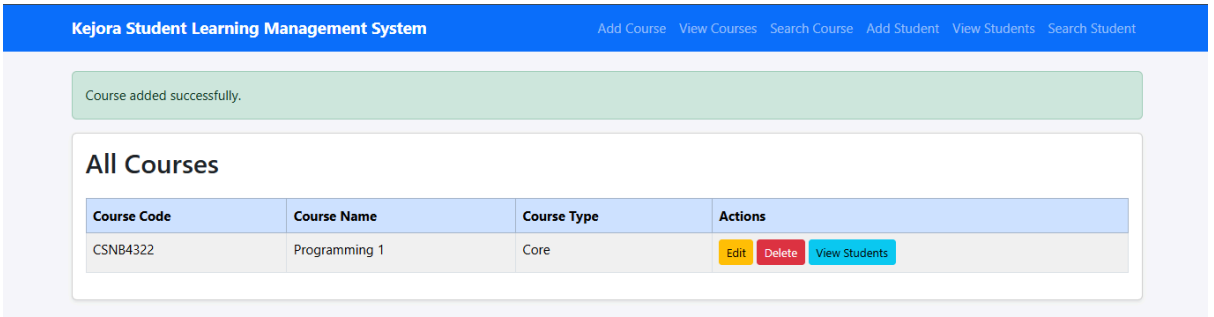
Adding a new student using the student id of another registered student is prohibited because to avoid duplication

SYSTEM TESTING FOR KSLMS

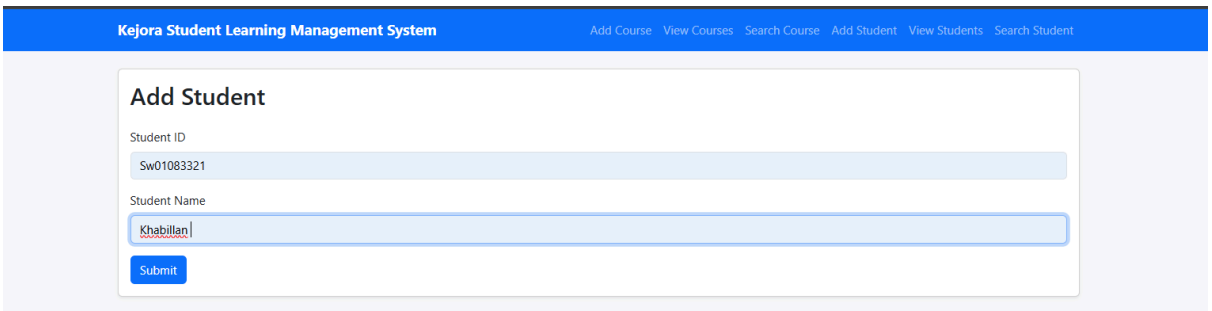
System Test Case 1: Full System Flow (Student → Course → API → Enrollment)



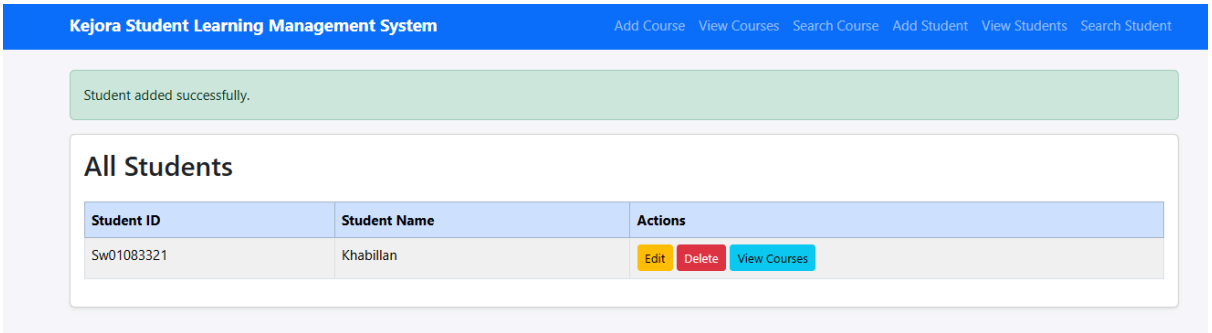
Add Course



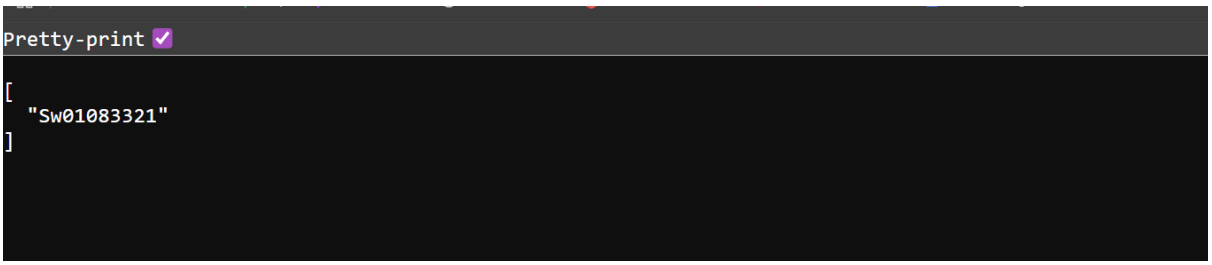
View the Course that is added



Add Student



View Student that is added

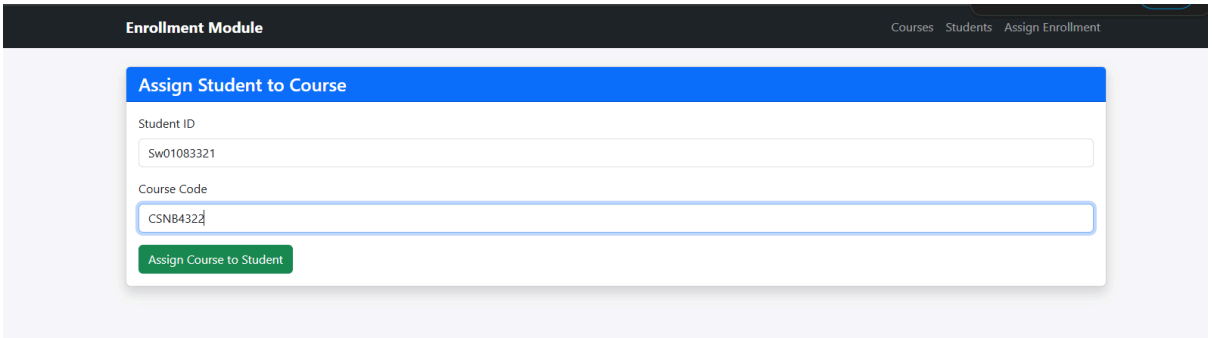


Student Id That is registered will be fetched by the API in Laravel

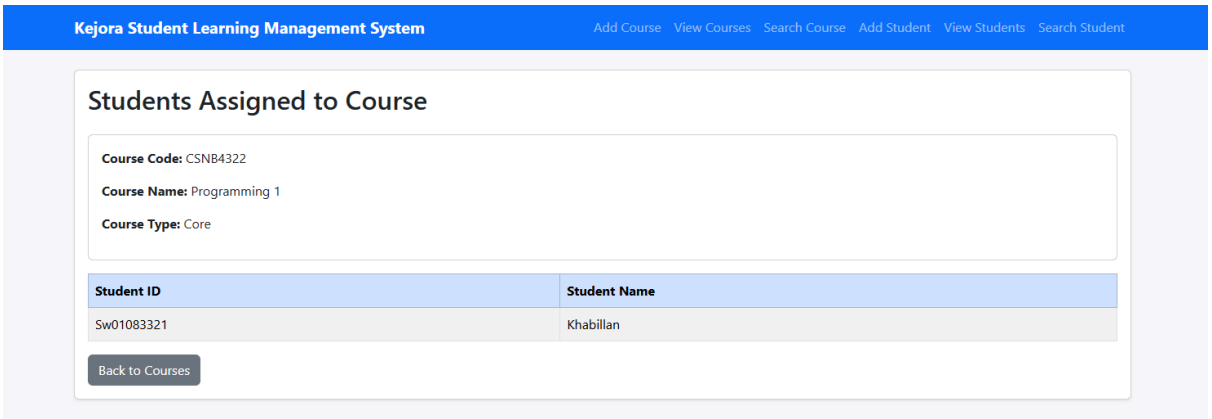
```
Pretty-print ☑

[
  "CSNB4322"
]
```


Course Code that is registered will be fetched by the API in Laravel

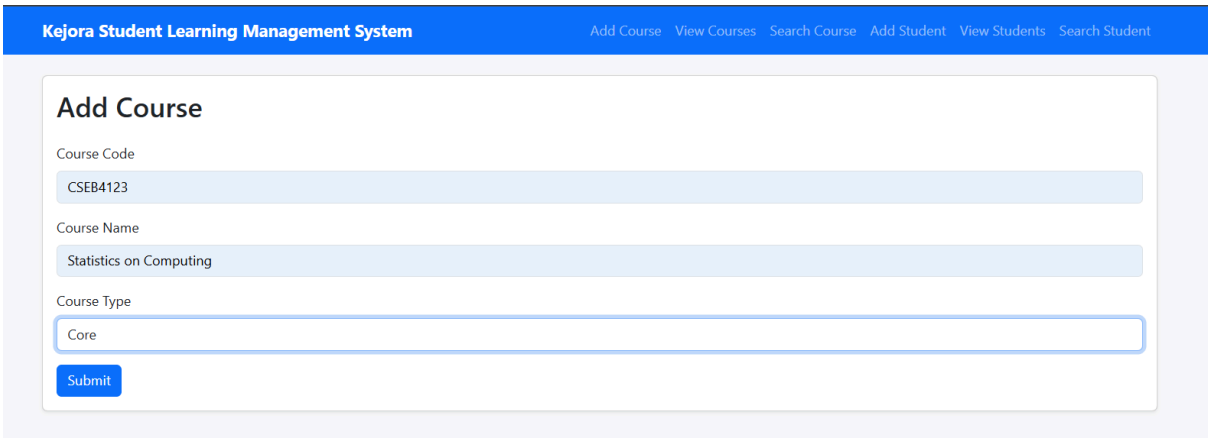


Assigning the student and the course in the enrollment module

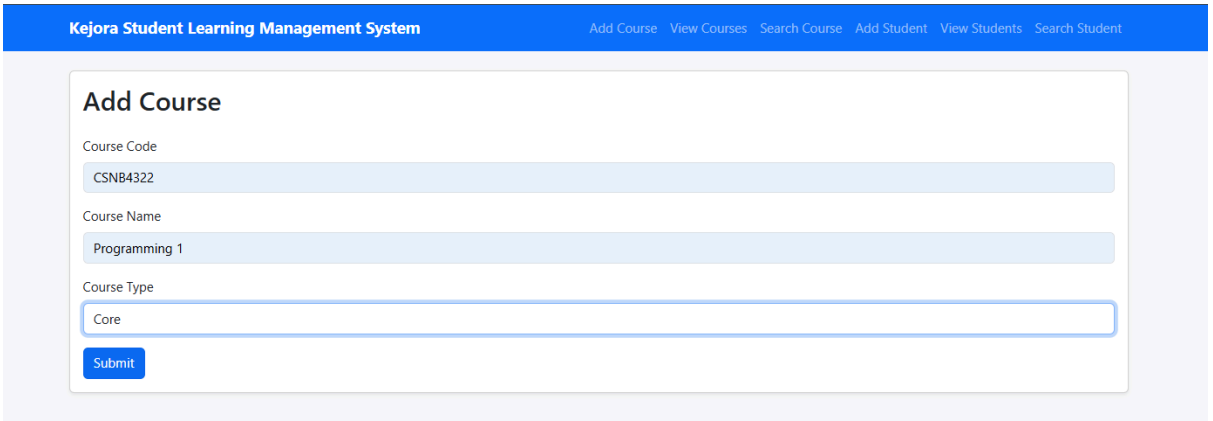


Successfully assigned because both are them are registered and fetched by Laravel Internal API to enrollment module to enroll the students and the course

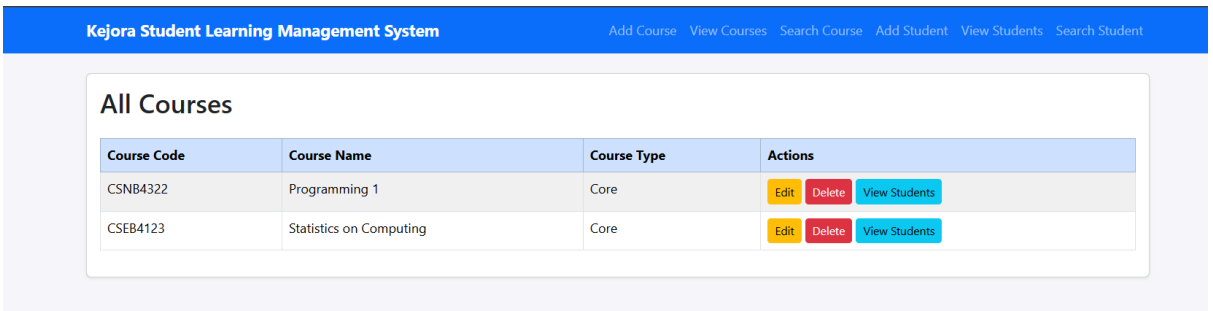
System Test Case 2: Enrolling Student to Course more than One and Vice Versa (Student → Course → API → Enrollment)



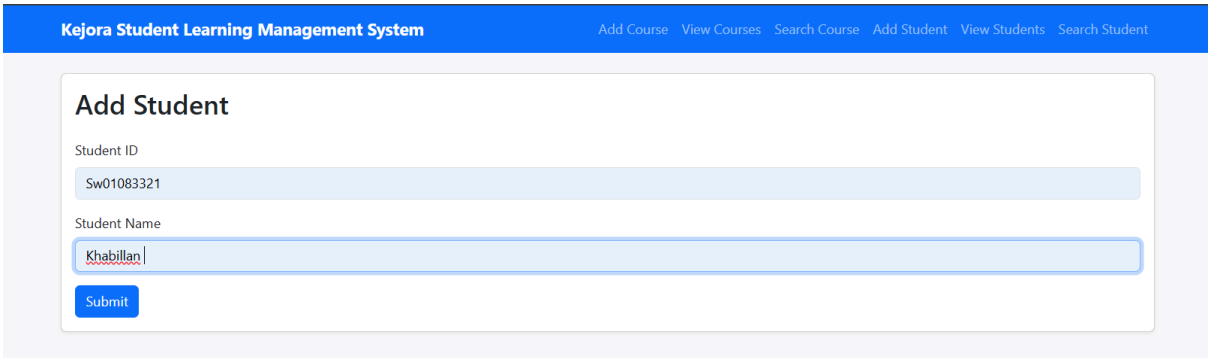
Add course (Statistics On Computing)



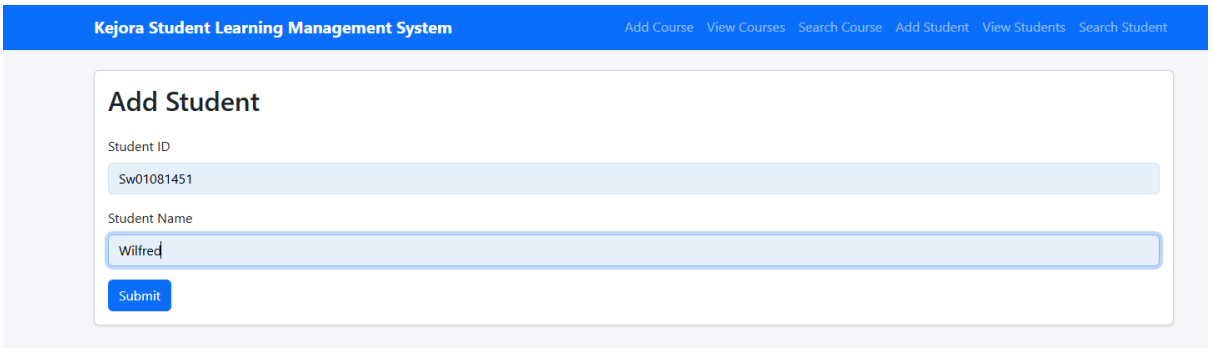
Add course (Programming 1)



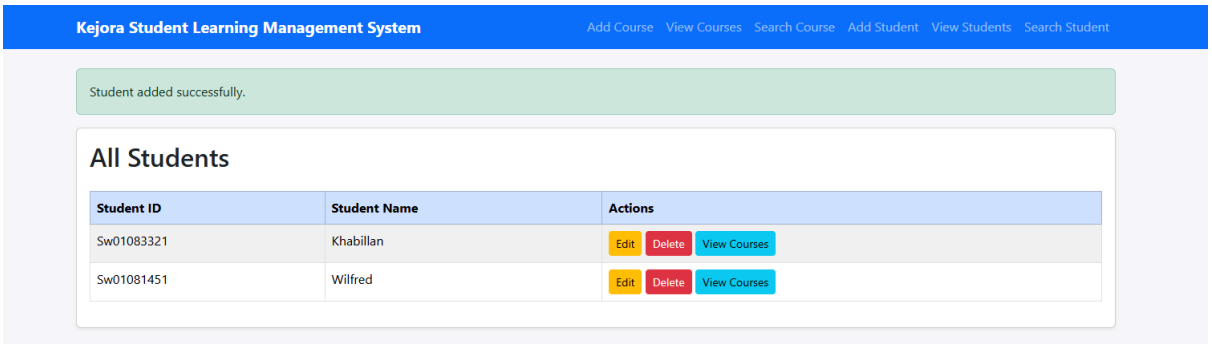
View Course That is Registered



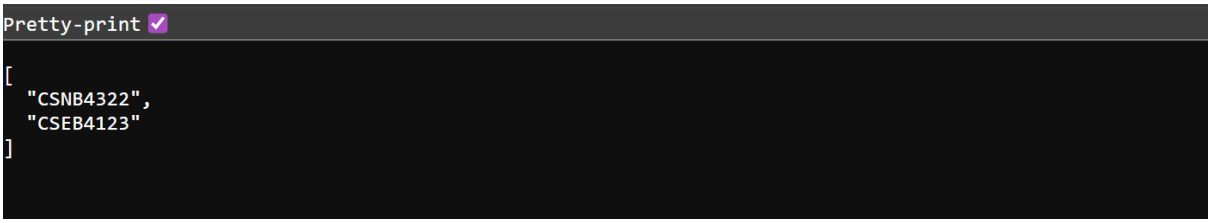
Add Student (Khabillan)



Add Student (Wilfred)

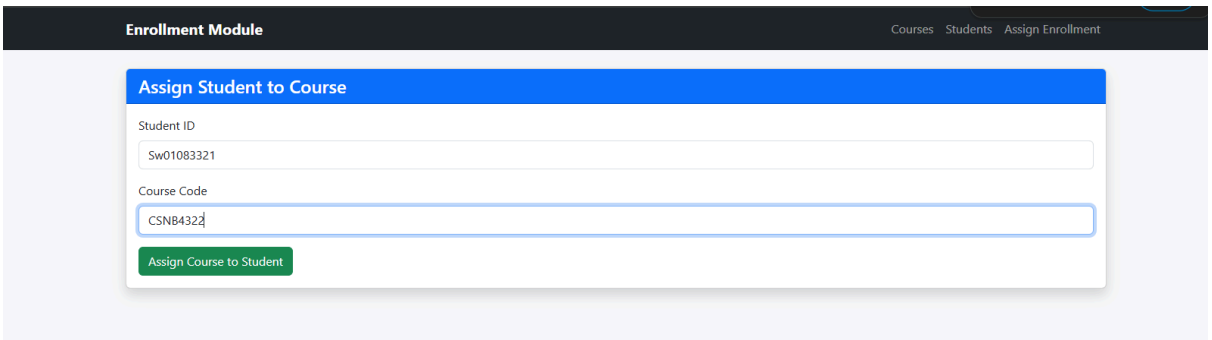


View student that is registered.

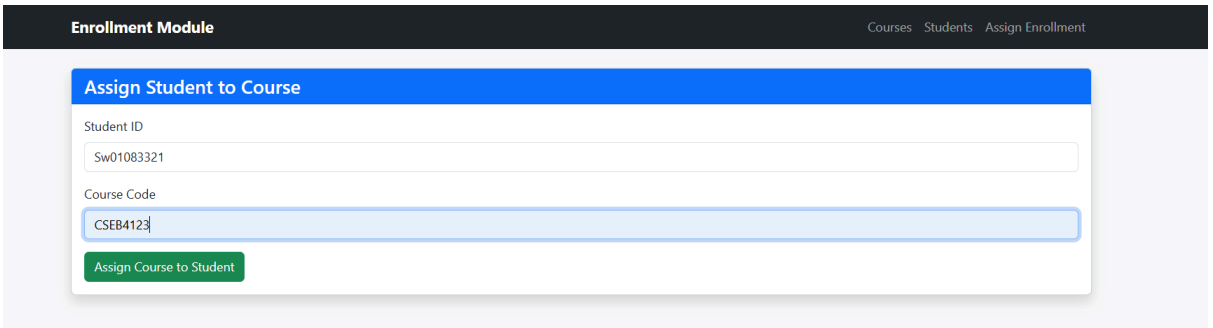


Course Code that is registered will be fetched by the API in Laravel . In here there are two course codes that are registered which are fetched by the API itself.

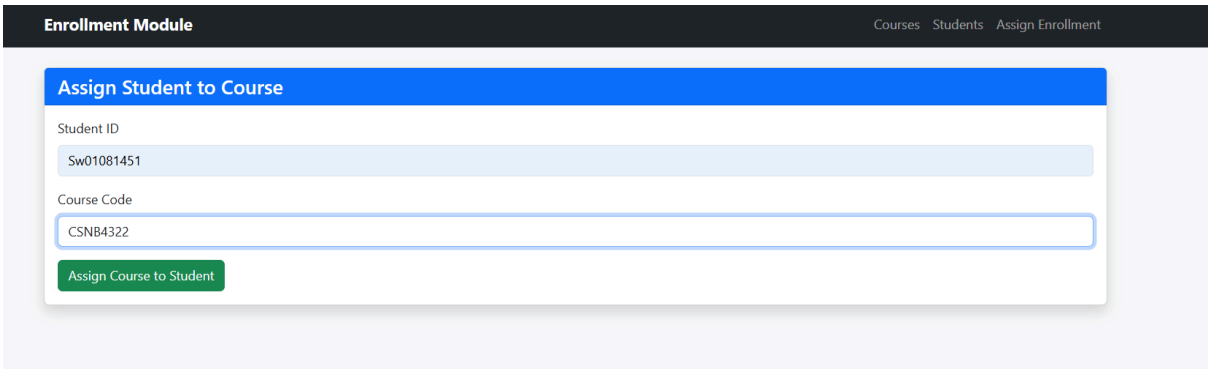
Student Id that is registered will be fetched by the API in Laravel. In here two student it which is registered are being fetched by the API



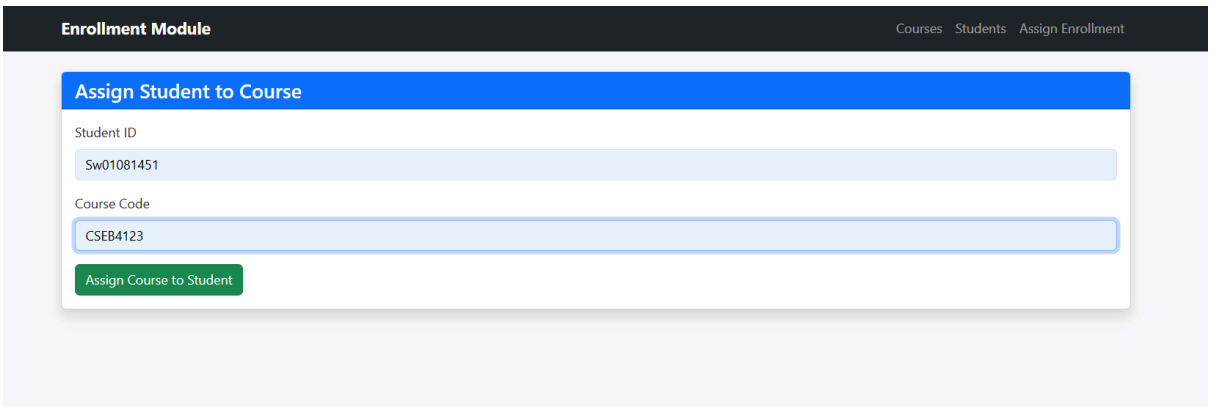
Assigning the student(Khabillan) to the course(CSNB4322) in the enrollment module



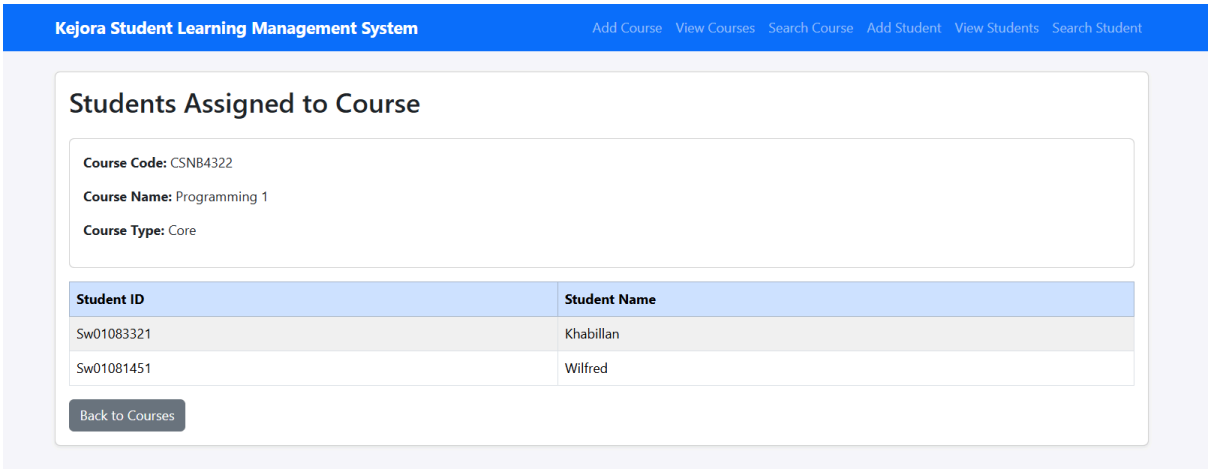
Assigning the student(Khabillan) to the course(CSEB4123) in the enrollment module



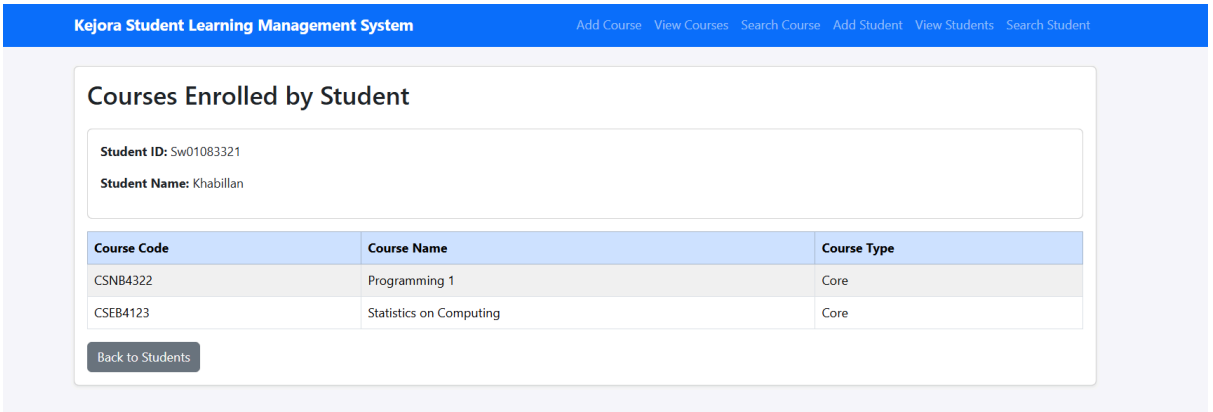
Assigning the student(Wilfred) to the course(CSNB4322) in the enrollment module



Assigning the student(Wilfred) to the course(CSEB4123) in the enrollment module

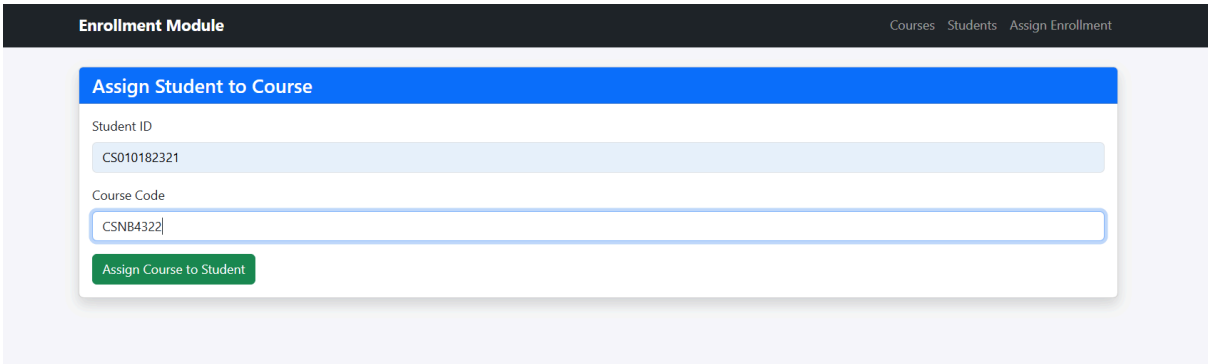


Two students successfully enrolled in one course

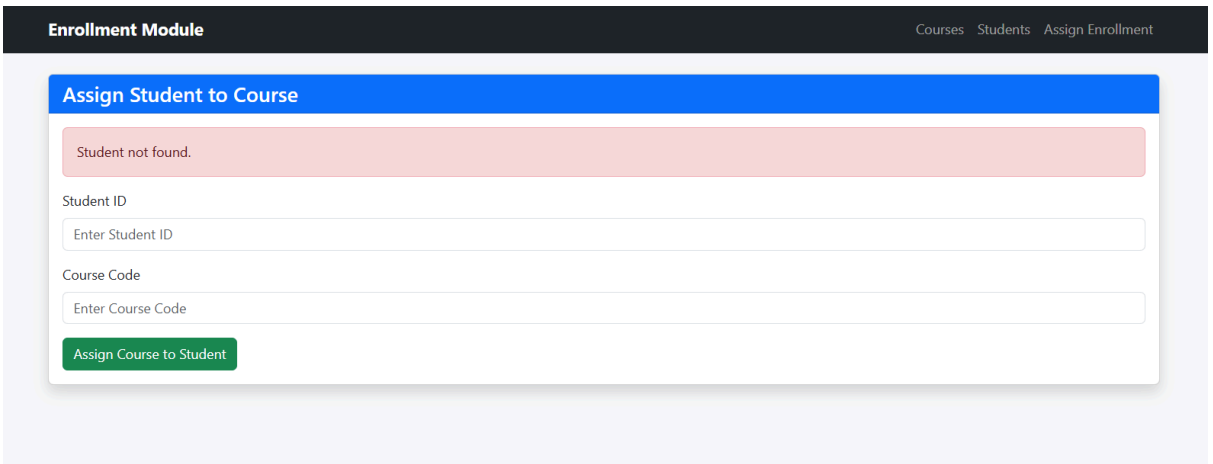


One student successfully enrolled in two courses

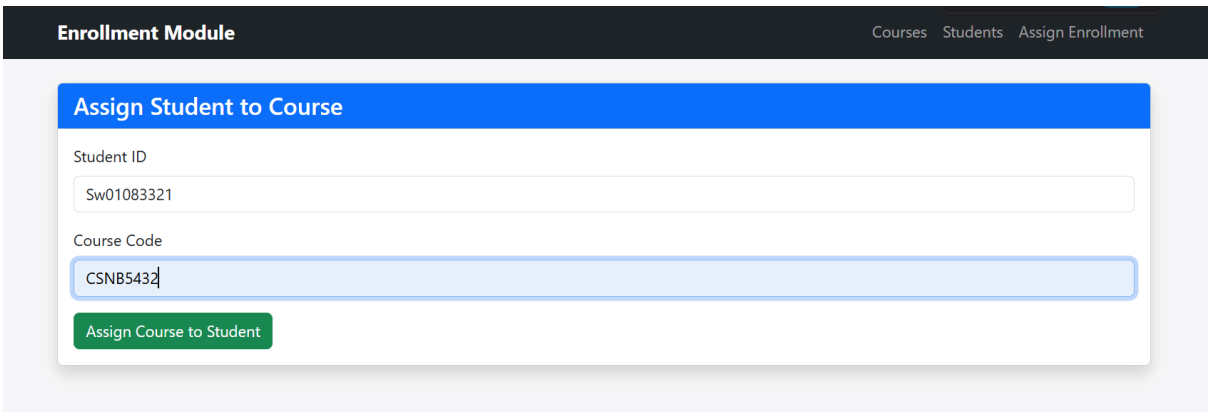
System Test Case 3: Invalid Enrollment Input



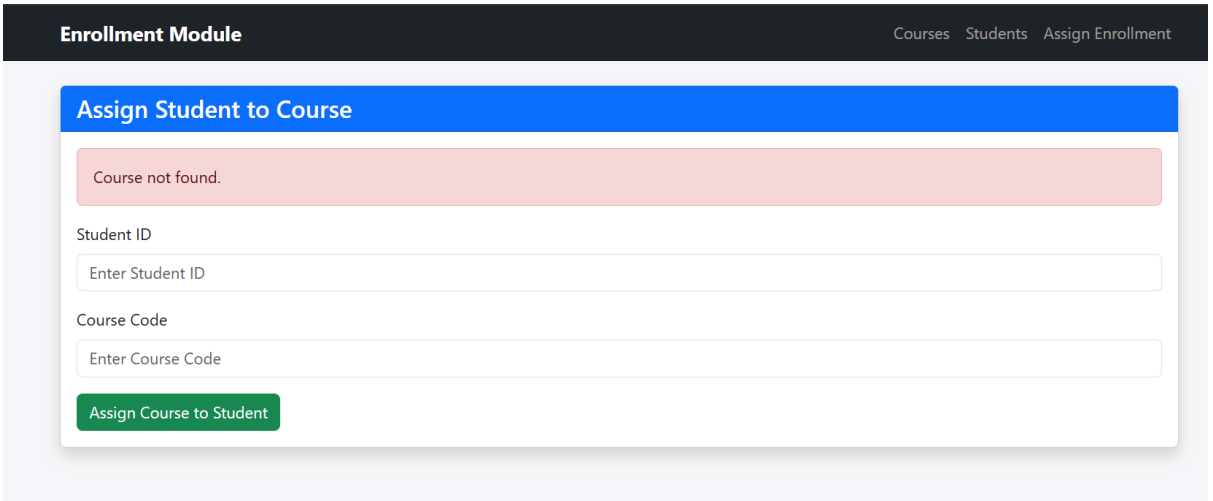
Random student id entered into the enrollment module



Unregistered student id is not accepted to enrol with course and it will show invalid response. This is because , API couldn't fetch student id that is not registered in the system

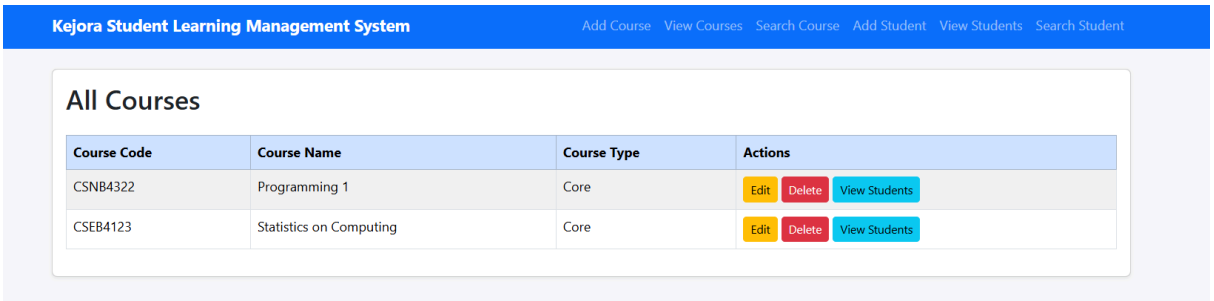


Random Course selected to enrol with the registered student

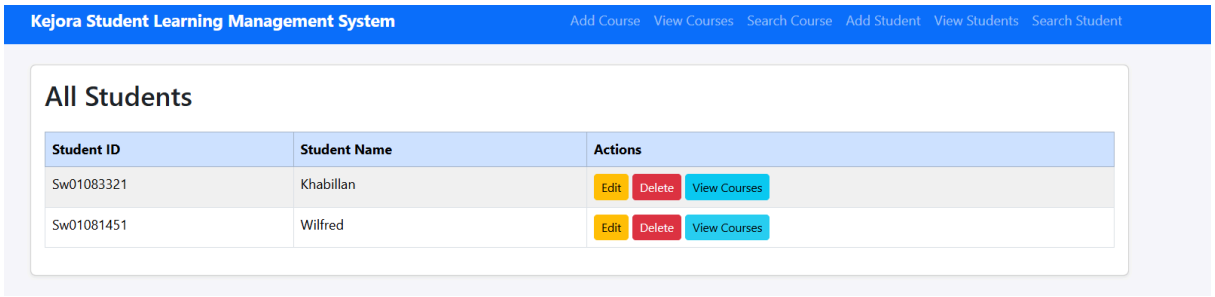


Unregistered course code is not accepted to enrol with student and it will show invalid response. This is because , API couldn't fetch course code that is not registered in the system.

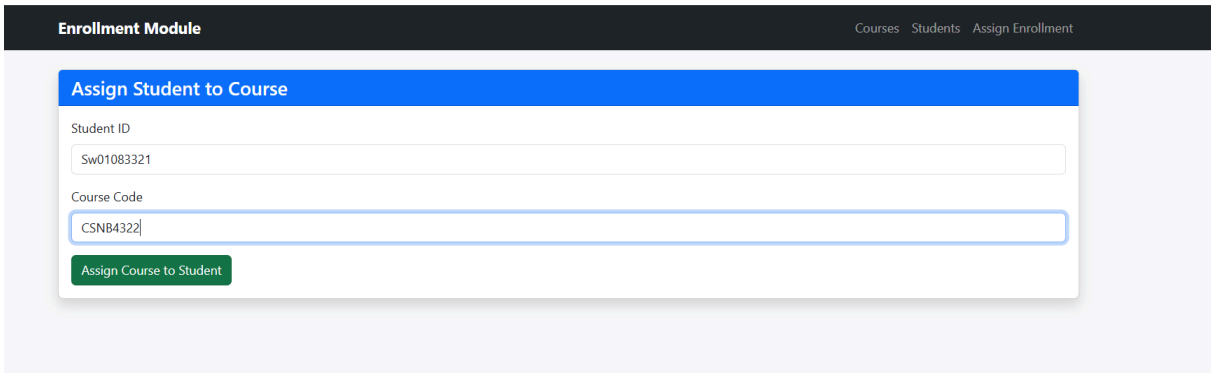
System Test Case 4: Duplicate Enrollment Check



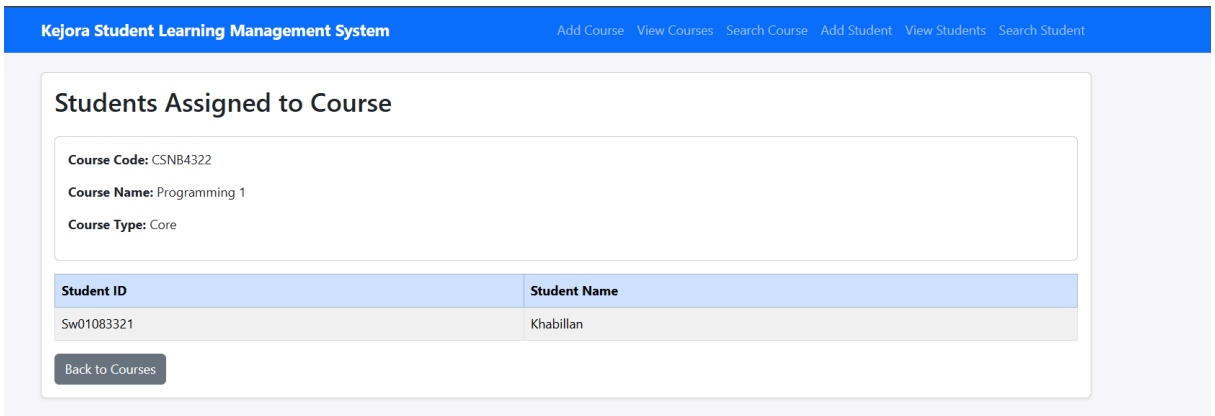
Courses that is Registered



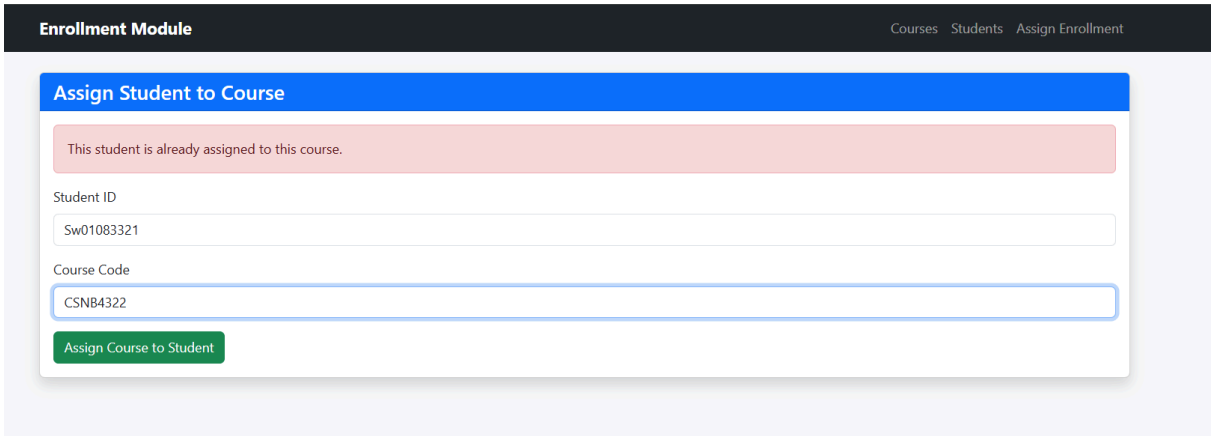
Student that is Registered



Enrolling a Student(Khabillan) on a course (CSNB4322)



Student is successfully enrolled



The System does not allow Student that is already enrolled to the course to enroll with it again to avoid duplication

SOFTWARE CONSTRUCTION BEST PRACTICES

In this lab, several software construction best practices were applied to ensure that the KSLMS system is well-structured, reliable, and easy to maintain. Firstly, the system follows a modular design by separating the Course, Student, and Enrollment functionalities into different controllers and views. This modular approach improves code organization and makes debugging and testing easier, as each module handles a specific responsibility.

Secondly, Object-Oriented Programming (OOP) principles were implemented through the use of the Course and Student classes. These classes use encapsulation by defining attributes such as course code and student ID as private and accessing them through getter methods. This ensures controlled access to data and maintains data integrity within the system. The unit testing results confirm that these getter methods work correctly when retrieving stored values.

Another important practice is defensive programming. The system includes validation checks to ensure that users cannot input invalid or duplicate data. For example, duplicate course codes and student IDs are not allowed, and appropriate error messages are displayed when such cases occur. Similarly, the search functions return “Course not found” or “Student not found” when invalid data is entered. This improves system robustness and user experience.

Furthermore, integration between modules is well implemented. The system uses Laravel API routes to fetch student IDs and course codes in JSON format, which are then used in the enrollment module. This ensures

that only valid and registered data is used when assigning students to courses. The successful system testing demonstrates that all modules Course, Student, API, and Enrollment are properly integrated and function together as a complete system.

CHALLENGES FACED DURING DEVELOPMENT

Several challenges were encountered during the development and testing of the KSLMS system. One major challenge was ensuring that the API correctly retrieves only registered student IDs and course codes from the system. When unregistered data is entered, the system must correctly identify it and prevent invalid enrollment. This required careful handling of session data and validation logic.

Another challenge was handling duplicate data. The system needed to prevent duplicate course codes, duplicate student IDs, and duplicate enrollment assignments. For example, assigning the same student to the same course more than once must be rejected. Implementing these checks required additional logic to verify existing records before adding new data.

Testing the system with both valid and invalid inputs was also challenging. The system had to be tested not only for correct inputs but also for error-prone scenarios such as empty fields, random IDs, and unregistered data. Ensuring that the system responds with appropriate error messages in all these cases was important to demonstrate proper defensive programming.

Finally, integrating all modules was another challenge. The Course, Student, API, and Enrollment modules needed to work seamlessly as one system. Any inconsistency in data or function behaviour could cause errors during enrollment. Through continuous testing and debugging, these integration issues were resolved, resulting in a stable and functional system.

CONCLUSION

5.1 Summary

The KSLMS system successfully demonstrates a structured approach to managing academic data using Laravel. The implementation of MVC architecture, OOP principles, and validation techniques ensures a reliable and maintainable system.

5.2 Key Learnings

- Importance of structured architecture (MVC)
- Benefits of encapsulation in OOP
- Role of validation in system reliability
- Handling data safely without a database

5.3 Future Improvements

- Integration with a database (MySQL)
- Advanced search algorithms
- User authentication system
- Improved UI/UX design